Double-click (or enter) to edit

Step 0: Environment Setup & Drive Mounting

```
import os
import numpy as np
import nibabel as nib
import torch
```

```
PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"

print("Exists:", os.path.exists(PREP_ROOT))
print("Folders:", os.listdir(PREP_ROOT)[:20])
```

```
Exists: True
Folders: ['_checkpoints']
```

```
import os
print("exists:", os.path.exists("/content/drive"))
if os.path.exists("/content/drive"):
    print(os.listdir("/content/drive")[:20])
```

```
exists: True
['MyDrive']
```

```
import shutil
import os

if os.path.exists("/content/drive"):
    shutil.rmtree("/content/drive")

print("After delete, exists:", os.path.exists("/content/drive"))
```

```
After delete, exists: False
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

```
import os

PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"
print("Exists:", os.path.exists(PREP_ROOT))
print("Folders:", os.listdir(PREP_ROOT)[:20])
```

```
Exists: True
Folders: ['00713519', '00960975', '00981157', '_visual_results', '_checkpoints', '01092669', '01386182', '01406050', '01722254',
```

```
import os
import numpy as np
import nibabel as nib
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
```

Step 1: Dataset Overview & Patient ID Discovery

```
import os, glob

DATA_ROOTS = [
    "/content/drive/MyDrive/14 MS patients",
    "/content/drive/MyDrive/50 patients"
]

patients = []

for root in DATA_ROOTS:
```

```
    if os.path.exists(root):
        found = [
            p for p in glob.glob(os.path.join(root, "*"))
            if os.path.isdir(p)
        ]
        patients.extend(found)

patients = sorted(patients)

print("Total patient folders found:", len(patients))
print("\nFirst few patient folders:")
for p in patients[:10]:
    print(" -", os.path.basename(p))
```

```
Total patient folders found: 0

First few patient folders:
```

Get list of patients from the 50 folder only

```
import os, glob

FIFTY_ROOT = "/content/drive/MyDrive/50 patients"

patients_50 = sorted([
    os.path.basename(p)
    for p in glob.glob(os.path.join(FIFTY_ROOT, "*"))
    if os.path.isdir(p)
])

print("Total patients in 50 folder:", len(patients_50))
print("First 15:", patients_50[:15])
```

```
Total patients in 50 folder: 0
First 15: []
```

code to pick new patients from the 50 Patients folder

```
import os, glob

FIFTY_ROOT = "/content/drive/MyDrive/50 patients"

patients_50 = sorted([
    os.path.basename(p)
    for p in glob.glob(os.path.join(FIFTY_ROOT, "*"))
    if os.path.isdir(p)
])

# Your CURRENT full train list (20) — paste your real list here
TRAIN_IDS_BASE = [
    "00713519", "01092669", "01386182", "01406050", "01722254",
    "01406050_1", "01563167", "01581148_2", "01581148_4", "01668999",
    "00516779", "00749499", "00832617", "00890265", "00890509",
    "00907490", "00952337", "00967329", "00967939", "01001835",
]

# Your CURRENT test list (7) — paste your real list here
TEST_IDS_BASE = [
    "00960975", "00981157", "01003818", "01019493",
    "01036141", "01083481", "01135143",
]

# 1) skip the bad one
BAD_ID = "01001835"
TRAIN_IDS_BASE = [x for x in TRAIN_IDS_BASE if x != BAD_ID]

IGNORE_IDS = {"00408990"}  # keep your ignore
USED = set(TRAIN_IDS_BASE + TEST_IDS_BASE + [BAD_ID])

# candidates from 50 folder that are not already used, not ignored, and not "(1)" duplicates
candidates = [
    p for p in patients_50
    if (p not in USED)
    and (p not in IGNORE_IDS)
    and ("(" not in p)
```

```
]

print("Candidate count (50 folder, clean):", len(candidates))
print("First 20 candidates:", candidates[:20])

# 2) add only ONE replacement
NEW_ID = candidates[0]
TRAIN_IDS = TRAIN_IDS_BASE + [NEW_ID]
TEST_IDS = TEST_IDS_BASE

print("\n✅ Removed BAD_ID:", BAD_ID)
print("✅ Replacement NEW_ID:", NEW_ID)
print("✅ Total TRAIN:", len(TRAIN_IDS), TRAIN_IDS)
print("✅ Total TEST :", len(TEST_IDS), TEST_IDS)
```

```
Candidate count (50 folder, clean): 0
First 20 candidates: []
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/tmp/ipykernel_9382/2867500910.py in <cell line: 0>()
     42
     43 # 2) add only ONE replacement
---> 44 NEW_ID = candidates[0]
     45 TRAIN_IDS = TRAIN_IDS_BASE + [NEW_ID]
     46 TEST_IDS = TEST_IDS_BASE

IndexError: list index out of range
```

```
import os

def check_patient(pid, root_dir):
    patient_dir = os.path.join(root_dir, pid)

    if not os.path.exists(patient_dir):
        print("❌ Patient folder not found:", patient_dir)
        return

    print("✅ Patient found:", patient_dir)

    files = os.listdir(patient_dir)
    print("\nFiles inside patient folder:")

    for f in files:
        print(" -", f)
```

```
check_patient("01137872", "/content/drive/MyDrive/50 patients")
```

quick safety check

```
print("Overlap train/test:", set(TRAIN_IDS) & set(TEST_IDS_BASE))
```

Verifying the ids

```
print("Training IDs:", TRAIN_IDS)
print("Testing IDs:", TEST_IDS)
```

Step 3: Inspect Selected Patient Folders

```
# ================================
# STEP 3 — Inspect chosen patients
# Identify baseline / follow-up / label files
# Works with TWO folders: 14 MS patients + 50 patients
# ================================

import os, re, glob
from pathlib import Path

# ✅ IMPORTANT:
```

```
# Do NOT redefine TRAIN_IDS / TEST_IDS here.
# This cell will use the TRAIN_IDS and TEST_IDS you already created in Step 2.
print("Training patients (count):", len(TRAIN_IDS))
print("Testing patients  (count):", len(TEST_IDS))
print("\nTRAIN_IDS:", TRAIN_IDS)
print("TEST_IDS :", TEST_IDS)

# ✅ Two data roots (both folders mounted under the same Drive)
DATA_ROOTS = [
    "/content/drive/MyDrive/14 MS patients",
    "/content/drive/MyDrive/50 patients"
]

def get_patient_dir(pid: str):
    """Return the first folder path that exists for this patient ID across DATA_ROOTS."""
    for root in DATA_ROOTS:
        pdir = os.path.join(root, pid)
        if os.path.isdir(pdir):
            return pdir
    return None

def parse_date_from_name(name: str):
    """
    Extract an 8-digit date (YYYYMMDD) from filename if present.
    Example: 00713519_20181002.nii.gz -> 20181002
    Returns int date or None.
    """
    m = re.search(r"(19|20)\d{6}", name)
    return int(m.group(0)) if m else None

def classify_files(patient_dir: str):
    """
    Returns:
      - baseline_path (earliest dated nii)
      - followup_path (latest dated nii)
      - label_path (best guess by name)
      - other_nii (everything else)
      - mats (mat files)
    """
    nii = sorted(glob.glob(os.path.join(patient_dir, "*.nii.gz")))
    mats = sorted(glob.glob(os.path.join(patient_dir, "*.mat")))

    # ----------------------------
    # Label candidates by name
    # ----------------------------
    label_candidates = []
    for f in nii:
        bn = os.path.basename(f).lower()
        if ("label" in bn) or ("seg" in bn) or ("mask" in bn) or ("gt" in bn):
            label_candidates.append(f)

    # Choose label: prefer anything containing "segmentation-label"
    label = None
    if label_candidates:
        exact = [f for f in label_candidates if "segmentation-label" in os.path.basename(f).lower()]
        label = exact[0] if exact else label_candidates[0]

    # Flair candidates exclude label files
    flair_candidates = [f for f in nii if f != label]

    # ----------------------------
    # Baseline / Follow-up by date
    # ----------------------------
    dated = []
    for f in flair_candidates:
        d = parse_date_from_name(os.path.basename(f))
        if d is not None:
            dated.append((d, f))

    baseline = followup = None
    if len(dated) >= 2:
        dated.sort(key=lambda x: x[0])
        baseline = dated[0][1]
        followup = dated[-1][1]
    else:
        # fallback: first two nifti
        if len(flair_candidates) >= 2:
```

```
            baseline, followup = flair_candidates[0], flair_candidates[1]

    other_nii = [f for f in nii if f not in [baseline, followup, label] and f is not None]

    return baseline, followup, label, other_nii, mats

def print_patient_summary(pid: str):
    pdir = get_patient_dir(pid)

    print("\n" + "="*80)
    print("Patient:", pid)
    print("Folder :", pdir if pdir else "NOT FOUND")

    if pdir is None:
        print("❌ Folder not found in any DATA_ROOTS.")
        return

    baseline, followup, label, other_nii, mats = classify_files(pdir)

    # List all files first
    all_files = sorted([p.name for p in Path(pdir).glob("*")])
    print("\nAll files in folder:")
    for f in all_files:
        print("  -", f)

    print("\n✅ Detected files:")
    if baseline:
        print("  Baseline  :", os.path.basename(baseline))
    else:
        print("  Baseline  : ⚠️ not detected")

    if followup:
        print("  Follow-up :", os.path.basename(followup))
    else:
        print("  Follow-up : ⚠️ not detected")

    if label:
        print("  Label/GT  :", os.path.basename(label))
    else:
        print("  Label/GT  : ⚠️ not detected (no file with label/seg/mask/gt in name)")

    if mats:
        print("\nMAT files (registration?):")
        for m in mats:
            print("  -", os.path.basename(m))
    else:
        print("\nMAT files: none")

    if other_nii:
        print("\nOther NIfTI files (maybe registered volumes, masks, etc.):")
        for f in other_nii:
            print("  -", os.path.basename(f))
    else:
        print("\nOther NIfTI files: none")

# ==============================
# Run summaries for your split
# ==============================
for pid in TRAIN_IDS + TEST_IDS:
    print_patient_summary(pid)
```

Sanity Check: Geometry & GT Alignment (Before Preprocessing)

```
import os, re
import nibabel as nib
import numpy as np

# ✅ Use your REAL lists (20 train + 7 test)
TRAIN_IDS = ['00713519', '01092669', '01386182', '01406050', '01722254',
             '01406050_1', '01563167', '01581148_2', '01581148_4', '01668999',
             '00516779', '00749499', '00832617', '00890265', '00890509',
             '00907490', '00952337', '00967329', '00967939', '01137872']

TEST_IDS  = ['00960975', '00981157', '01003818', '01019493', '01036141',
             '01083481', '01135143']
```

```
def load_info(path):
    img = nib.load(path)
    return img.shape, img.header.get_zooms()[:3], img.affine

def affine_close(a, b, tol=1e-3):
    return np.allclose(a, b, atol=tol)

def pick_patient_files(pdir):
    files = [f for f in os.listdir(pdir) if f.endswith(".nii.gz")]

    # 1) label (handles Segmentation-label, Segmentation-label_2, Segmentation-Segment_1-label, etc.)
    label_candidates = [f for f in files if f.lower().startswith("segmentation") and "label" in f.lower()]
    if not label_candidates:
        raise FileNotFoundError("No segmentation label file found")
    label = sorted(label_candidates)[0]

    # 2) registered follow-up (your dataset uses _to_FLAIR_1 OR _to_FLAIR1)
    reg_candidates = [f for f in files if re.search(r"_to_FLAIR_?1\.nii\.gz$", f)]
    if not reg_candidates:
        raise FileNotFoundError("No registered follow-up '*_to_FLAIR_1.nii.gz' or '*_to_FLAIR1.nii.gz' found")
    follow_reg = sorted(reg_candidates)[0]

    # 3) dated scans (original baseline + original follow-up)
    # exclude label + registered
    dated = []
    for f in files:
        if f == label:
            continue
        if f == follow_reg:
            continue
        # find YYYYMMDD inside filename
        m = re.search(r"(19|20)\d{6}", f)
        if m:
            dated.append((int(m.group()), f))

    if len(dated) < 2:
        raise FileNotFoundError("Could not find 2 dated scans for baseline & follow-up")

    dated.sort()
    baseline = dated[0][1]
    follow_orig = dated[-1][1]

    return baseline, follow_orig, follow_reg, label

def check_patient(pid, data_root):
    pdir = os.path.join(data_root, pid)
    baseline, follow_orig, follow_reg, label = pick_patient_files(pdir)

    paths = {
        "baseline": os.path.join(pdir, baseline),
        "followup_original": os.path.join(pdir, follow_orig),
        "followup_registered": os.path.join(pdir, follow_reg),
        "label": os.path.join(pdir, label),
    }

    print("\n" + "="*100)
    print("Patient:", pid)
    for k, p in paths.items():
        sh, z, _ = load_info(p)
        print(f"{k:20s} | shape={sh} | zooms={z} | file={os.path.basename(p)}")

    b_sh, _, b_aff = load_info(paths["baseline"])
    r_sh, _, r_aff = load_info(paths["followup_registered"])
    l_sh, _, l_aff = load_info(paths["label"])

    print("\nGeometry checks:")
    print("Registered follow-up vs BASELINE:")
    print("  same shape? ", r_sh == b_sh)
    print("  same affine? ", affine_close(r_aff, b_aff))

    print("Label vs BASELINE:")
    print("  same shape? ", l_sh == b_sh)
    print("  same affine? ", affine_close(l_aff, b_aff))

    print("Label vs REGISTERED follow-up:")
    print("  same shape? ", l_sh == r_sh)
```

```
    print("  same affine? ", affine_close(l_aff, r_aff))

# ✅ Set the correct root(s)
# If you truly have 2 roots (14 MS patients vs 50 patients), you can run twice.
ROOTS = [
    "/content/drive/MyDrive/14 MS patients",
    "/content/drive/MyDrive/50 patients",
]

# ✅ Run checks
for root in ROOTS:
    print("\n" + "#"*100)
    print("Checking root:", root)
    for pid in TRAIN_IDS + TEST_IDS:
        pdir = os.path.join(root, pid)
        if os.path.isdir(pdir):
            try:
                check_patient(pid, root)
            except Exception as e:
                print("\n" + "="*100)
                print("Patient:", pid)
                print("ERROR:", e)
```

Preprocessing

Step 4: Preprocessing – Global Target Shape Definition

```
import os, re, glob
import nibabel as nib
import numpy as np
from math import ceil

# ================================
# ✅ BEST RECOMMENDATION:
# Compute TARGET_SHAPE using ONLY your TRAIN + TEST patients
# across BOTH roots (14 MS patients + 50 patients),
# then round up to a multiple of 16 (safer for UNet downsampling).
# ================================

ROOTS = [
    "/content/drive/MyDrive/14 MS patients",
    "/content/drive/MyDrive/50 patients",
]

TARGET_SPACING = (0.5, 0.5, 0.5)

# ✅ your final lists (use the corrected ones)
TRAIN_IDS = ['00713519', '01092669', '01386182', '01406050', '01722254',
             '01406050_1', '01563167', '01581148_2', '01581148_4', '01668999',
             '00516779', '00749499', '00832617', '00890265', '00890509',
             '00907490', '00952337', '00967329', '00967939', '01137872']

TEST_IDS  = ['00960975', '00981157', '01003818', '01019493', '01036141',
             '01083481', '01135143']

# Explicitly ignore known bad patients
IGNORE_IDS = {"00408990"}

def parse_date_from_name(name: str):
    m = re.search(r"(19|20)\d{6}", name)
    return int(m.group(0)) if m else None

def pick_baseline_and_registered(patient_dir: str):
    """
    Returns:
      baseline_path, registered_followup_path
    Works for both naming styles:
      - *_to_FLAIR_1.nii.gz
      - *_to_FLAIR1.nii.gz
    """
    nii = [f for f in glob.glob(os.path.join(patient_dir, "*.nii.gz"))]
    if not nii:
        return None, None

    # label files (exclude)
```

```
    def is_label(f):
        bn = os.path.basename(f).lower()
        return bn.startswith("segmentation") and "label" in bn

    # registered followup candidates
    reg = [f for f in nii if re.search(r"_to_FLAIR_?1\.nii\.gz$", os.path.basename(f))]
    if not reg:
        return None, None
    reg_path = reg[0]

    # baseline = earliest dated scan among non-label and not registered
    dated = []
    for f in nii:
        if f == reg_path:
            continue
        if is_label(f):
            continue
        d = parse_date_from_name(os.path.basename(f))
        if d is not None:
            dated.append((d, f))

    if len(dated) < 1:
        return None, None

    dated.sort(key=lambda x: x[0])
    baseline_path = dated[0][1]
    return baseline_path, reg_path

def estimate_resampled_shape(shape, zooms, target_spacing):
    # shape and zooms are 3D
    return tuple(int(np.round(shape[i] * (zooms[i] / target_spacing[i]))) for i in range(3))

def round_up_to_multiple(x, m=16):
    return int(ceil(x / m) * m)

ALL_IDS = [pid for pid in (TRAIN_IDS + TEST_IDS) if pid not in IGNORE_IDS]

max_shape = (0, 0, 0)
used_volumes = []  # (pid, which, orig_shape, zooms, est_shape, root)

for pid in ALL_IDS:
    found = False
    for root in ROOTS:
        pdir = os.path.join(root, pid)
        if not os.path.isdir(pdir):
            continue

        baseline_path, reg_path = pick_baseline_and_registered(pdir)
        if baseline_path is None or reg_path is None:
            continue

        for which, path in [("baseline", baseline_path), ("followup_registered", reg_path)]:
            img = nib.load(path)
            shape = img.shape[:3]
            zooms = img.header.get_zooms()[:3]
            est = estimate_resampled_shape(shape, zooms, TARGET_SPACING)

            max_shape = tuple(max(max_shape[i], est[i]) for i in range(3))
            used_volumes.append((pid, which, shape, tuple(float(z) for z in zooms), est, root))

        found = True
        break  # stop after first root that contains this pid

    if not found:
        print(f"⚠️ WARNING: {pid} not found in any root (skipped)")

# ✅ round up to multiple of 16 for safer UNet pooling/upsampling
target_shape_rounded = tuple(round_up_to_multiple(s, 16) for s in max_shape)

print("\n==============================")
print("TARGET_SPACING:", TARGET_SPACING)
print("Computed from IDs only:", len(ALL_IDS), "(TRAIN+TEST)")
print("Total volumes used (baseline+reg across roots):", len(used_volumes))
print("TARGET_SHAPE (raw max):", max_shape)
print("TARGET_SHAPE (rounded to x16):", target_shape_rounded)

print("\nLargest estimated resampled volumes (top 10):")
```

```
used_volumes_sorted = sorted(
    used_volumes,
    key=lambda x: x[4][0]*x[4][1]*x[4][2],
    reverse=True
)[:10]

for pid, which, shape, zooms, est, root in used_volumes_sorted:
    print(f"{pid:8s} | {which:17s} | orig={shape} zooms={zooms} -> est={est} | root={os.path.basename(root)}")
```

Step 5.1(Training): Preprocessing Execution (NeuroPoly-style, Per Patient)

- Preprocess ONE training patient
- This will do, for TRAIN_ID=00713519:
- Load baseline + follow-up (original) + GT
- Register baseline → follow-up (follow-up reference, like NeuroPoly)
- Resample to 0.5mm (both images, plus label with nearest-neighbor)
- Normalize intensities
- (Optional) pad (we'll implement padding in a way that won't blow memory

```
print("TRAIN count:", len(TRAIN_IDS))
print("TEST count :", len(TEST_IDS))
print("Total      :", len(TRAIN_IDS) + len(TEST_IDS))
```

```
!pip -q install SimpleITK
```

```
# ================================
# Preprocess TRAIN patients (PAD ONLY, NO CROPPING)
# Uses existing registered follow-up (*_to_FLAIR1 / *_to_FLAIR_1)
# Resample to 0.5mm, then PAD to fixed TARGET_SHAPE, z-score in foreground.
# ================================

import os, re, glob
import numpy as np
import SimpleITK as sitk

# --------- ROOTS ----------
ROOTS = [
    "/content/drive/MyDrive/14 MS patients",
    "/content/drive/MyDrive/50 patients",
]

OUT_ROOT = "/content/drive/MyDrive/ms_preprocessed"
os.makedirs(OUT_ROOT, exist_ok=True)

# --------- SPLITS ----------
TRAIN_IDS = [
    '00713519','01092669','01386182','01406050','01722254',
    '01406050_1','01563167','01581148_2','01581148_4','01668999',
    '00516779','00749499','00832617','00890265','00890509',
    '00907490','00952337','00967329','00967939','01137872'
]

# If you also want to run test later, define TEST_IDS similarly.

TARGET_SPACING = (0.5, 0.5, 0.5)   # (x,y,z) in SimpleITK
TARGET_SHAPE   = (384, 512, 512)   # (x,y,z) PAD ONLY target

IGNORE_IDS = {"00408990"}

# --------- HELPERS ----------
def parse_date(name: str):
    m = re.search(r"(19|20)\d{6}", name)
    return int(m.group(0)) if m else None

def find_patient_dir(pid: str):
    for root in ROOTS:
        pdir = os.path.join(root, pid)
        if os.path.isdir(pdir):
            return pdir
```

```
    return None

def pick_patient_files(pid: str):
    if pid in IGNORE_IDS:
        raise FileNotFoundError(f"{pid} is ignored")

    pdir = find_patient_dir(pid)
    if pdir is None:
        raise FileNotFoundError(f"Patient folder not found in ROOTS: {pid}")

    nii = sorted(glob.glob(os.path.join(pdir, "*.nii.gz")))
    if not nii:
        raise FileNotFoundError(f"No .nii.gz files found in {pdir}")

    # 1) label (handles Segmentation-label, Segmentation-label_2, Segmentation-Segment_1-label, etc.)
    label_candidates = [
        f for f in nii
        if os.path.basename(f).lower().startswith("segmentation") and "label" in os.path.basename(f).lower()
    ]
    if not label_candidates:
        raise FileNotFoundError(f"No segmentation label file found for {pid}")
    label = sorted(label_candidates)[0]

    # 2) registered follow-up (your dataset uses _to_FLAIR_1 OR _to_FLAIR1)
    reg_candidates = [
        f for f in nii
        if re.search(r"_to_FLAIR_?1\.nii\.gz$", os.path.basename(f), flags=re.IGNORECASE)
    ]
    if not reg_candidates:
        raise FileNotFoundError(f"No registered follow-up '*_to_FLAIR1.nii.gz' or '*_to_FLAIR_1.nii.gz' for {pid}")
    follow_reg = reg_candidates[0]

    # 3) baseline = earliest dated scan (exclude label + follow_reg)
    dated = []
    for f in nii:
        if f == label or f == follow_reg:
            continue
        d = parse_date(os.path.basename(f))
        if d is not None:
            dated.append((d, f))

    if not dated:
        raise FileNotFoundError(f"No dated scan found for baseline for {pid}")

    dated.sort(key=lambda x: x[0])
    baseline = dated[0][1]

    return pdir, baseline, follow_reg, label

def resample_to_spacing(img, out_spacing, interpolator, default_value=0.0):
    """
    Resample to desired spacing (x,y,z) while preserving physical extent.
    """
    in_spacing = img.GetSpacing()
    in_size    = img.GetSize()  # (x,y,z)

    out_size = [
        int(np.round(in_size[i] * (in_spacing[i] / out_spacing[i])))
        for i in range(3)
    ]

    ref = sitk.Image(out_size, img.GetPixelID())
    ref.SetSpacing(out_spacing)
    ref.SetDirection(img.GetDirection())
    ref.SetOrigin(img.GetOrigin())

    return sitk.Resample(img, ref, sitk.Transform(), interpolator, default_value, img.GetPixelID())

def center_pad_only(img, target_size_xyz, pad_value=0.0):
    """Pad around center to target_size_xyz (x,y,z). NO CROPPING."""
    size = np.array(list(img.GetSize()), dtype=int)      # (x,y,z)
    target = np.array(list(target_size_xyz), dtype=int)

    # If any dimension is bigger than target -> would require cropping (not allowed)
    if np.any(size > target):
        raise ValueError(
            f"Resampled size {tuple(size)} exceeds TARGET_SHAPE {tuple(target)} "
```

```
            f"(cropping would be needed, not allowed)."
        )

    pad_lower = (target - size) // 2
    pad_upper = (target - size) - pad_lower

    # SimpleITK ConstantPad uses positional args in your version
    return sitk.ConstantPad(
        img,
        [int(p) for p in pad_lower.tolist()],
        [int(p) for p in pad_upper.tolist()],
        float(pad_value)
    )

def simple_foreground_mask(img):
    """
    Quick foreground mask: threshold above 5th percentile of non-zero voxels.
    """
    arr = sitk.GetArrayFromImage(img).astype(np.float32)
    nz = arr[arr > 0]
    if nz.size == 0:
        m = arr != 0
    else:
        thr = np.percentile(nz, 5)
        m = arr > thr
    out = sitk.GetImageFromArray(m.astype(np.uint8))
    out.CopyInformation(img)
    return out

def zscore_in_mask(vol, mask):
    v = sitk.GetArrayFromImage(vol).astype(np.float32)
    m = sitk.GetArrayFromImage(mask) > 0
    if m.sum() == 0:
        m = v != 0
    vals = v[m]
    mu = float(vals.mean())
    sd = float(vals.std() + 1e-6)
    v = (v - mu) / sd
    out = sitk.GetImageFromArray(v)
    out.CopyInformation(vol)
    return out

# ==============================
# RUN
# ==============================
print("TARGET_SPACING:", TARGET_SPACING)
print("TARGET_SHAPE  :", TARGET_SHAPE, "(PAD ONLY)")

for pid in TRAIN_IDS:
    print("\n" + "="*90)
    print("Preprocessing TRAIN patient:", pid)

    try:
        pdir, baseline_p, followreg_p, label_p = pick_patient_files(pid)
        print("Folder  :", pdir)
        print("Baseline:", os.path.basename(baseline_p))
        print("FollowR :", os.path.basename(followreg_p))
        print("Label   :", os.path.basename(label_p))

        # Read
        b  = sitk.ReadImage(baseline_p)
        fr = sitk.ReadImage(followreg_p)   # already registered follow-up in baseline space
        gt = sitk.ReadImage(label_p)

        # 1) Resample to 0.5mm
        b_05  = resample_to_spacing(b,  TARGET_SPACING, sitk.sitkLinear, 0.0)
        fr_05 = resample_to_spacing(fr, TARGET_SPACING, sitk.sitkLinear, 0.0)
        gt_05 = resample_to_spacing(gt, TARGET_SPACING, sitk.sitkNearestNeighbor, 0)

        # 2) PAD ONLY to fixed TARGET_SHAPE
        b_fix  = center_pad_only(b_05,  TARGET_SHAPE, pad_value=0.0)
        fr_fix = center_pad_only(fr_05, TARGET_SHAPE, pad_value=0.0)
        gt_fix = center_pad_only(gt_05, TARGET_SHAPE, pad_value=0)

        # 3) Z-score normalization using follow-up registered foreground mask
        mask = simple_foreground_mask(fr_fix)
        b_norm  = zscore_in_mask(b_fix,  mask)
```

```
        fr_norm = zscore_in_mask(fr_fix, mask)

        # 4) Save
        out_dir = os.path.join(OUT_ROOT, pid)
        os.makedirs(out_dir, exist_ok=True)

        sitk.WriteImage(b_norm,  os.path.join(out_dir, "baseline_0p5_norm_padded.nii.gz"))
        sitk.WriteImage(fr_norm, os.path.join(out_dir, "followup_registered_0p5_norm_padded.nii.gz"))
        sitk.WriteImage(gt_fix,  os.path.join(out_dir, "label_0p5_padded.nii.gz"))

        print("Saved to:", out_dir)
        print("Done ✅")

    except Exception as e:
        print(f"❌ Failed for {pid}: {e}")
```

We must sanity-check that:

- baseline_registered_to_followup and followup are aligned
- label is in the same follow-up space (very important)
- The label overlaps plausible bright lesion regions (at least visually)
- So the next step is: visualize a few slices (like your instructor's GT vs prediction style, but now as an overlay on the follow-up).

```
import os
import numpy as np
import nibabel as nib
import matplotlib.pyplot as plt

# ========================
# ✅ SPLITS (20 TRAIN + 7 TEST)
# ========================
TRAIN_IDS = [
    "00713519","01092669","01386182","01406050","01722254",
    "01406050_1","01563167","01581148_2","01581148_4","01668999",
    "00516779","00749499","00832617","00890265","00890509",
    "00907490","00952337","00967329","00967939","01137872",
]

TEST_IDS = [
    "00960975","00981157","01003818","01019493","01036141","01083481","01135143"
]

BASE_PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"

# ========================
# ✅ NEW (PAD-ONLY) OUTPUT NAMES
# ========================
F_NAME  = "followup_registered_0p5_norm_padded.nii.gz"
B_NAME  = "baseline_0p5_norm_padded.nii.gz"
GT_NAME = "label_0p5_padded.nii.gz"

def load_nii(path):
    img = nib.load(path)
    data = img.get_fdata().astype(np.float32)
    return data, img

def show_patient(pid, max_slices=3):
    prep_dir = os.path.join(BASE_PREP_ROOT, pid)
    print("\n" + "="*100)
    print("Patient:", pid)
    print("Folder :", prep_dir)

    f_path  = os.path.join(prep_dir, F_NAME)
    b_path  = os.path.join(prep_dir, B_NAME)
    gt_path = os.path.join(prep_dir, GT_NAME)

    # --- check existence ---
    missing = [p for p in [f_path, b_path, gt_path] if not os.path.exists(p)]
    if missing:
        print("❌ Missing files:")
        for m in missing:
            print(" -", os.path.basename(m))
        return
```

```
    # --- load ---
    f, _  = load_nii(f_path)
    b, _  = load_nii(b_path)
    gt, _ = load_nii(gt_path)

    gt_bin = gt > 0.5

    print("Shapes (F, B, GT):", f.shape, b.shape, gt_bin.shape)
    print("GT voxels:", int(gt_bin.sum()))

    # ========================
    # ✅ Slice along Z (axial): data[:, :, z]
    # ========================
    z_sums = gt_bin.sum(axis=(0, 1))          # sum over X,Y for each Z
    z_idx  = np.where(z_sums > 0)[0]

    if z_idx.size == 0:
        print("⚠️ No GT voxels found after preprocessing for this patient.")
        return

    # pick up to 3 slices between first and last GT slice
    if z_idx.size == 1:
        chosen = [int(z_idx[0])]
    else:
        chosen = np.linspace(z_idx.min(), z_idx.max(), max_slices).astype(int).tolist()

    print("Showing Z slices:", chosen)

    for z in chosen:
        fig, ax = plt.subplots(1, 3, figsize=(13, 4))

        # transpose for nicer viewing (imshow uses row/col)
        ax[0].imshow(f[:, :, z].T, cmap="gray", origin="lower")
        ax[0].set_title(f"{pid} Follow-up REG (z={z})")
        ax[0].axis("off")

        ax[1].imshow(b[:, :, z].T, cmap="gray", origin="lower")
        ax[1].set_title(f"{pid} Baseline (z={z})")
        ax[1].axis("off")

        ax[2].imshow(f[:, :, z].T, cmap="gray", origin="lower")
        ax[2].imshow(gt_bin[:, :, z].T, alpha=0.5, cmap="Reds", origin="lower")
        ax[2].set_title(f"{pid} Overlay GT (z={z})")
        ax[2].axis("off")

        plt.tight_layout()
        plt.show()

# ========================
# ✅ RUN FOR TRAIN + TEST
# ========================
print(f"TRAIN count: {len(TRAIN_IDS)}")
print(f"TEST  count: {len(TEST_IDS)}")
print(f"Total       : {len(TRAIN_IDS) + len(TEST_IDS)}")

# Show all patients (train then test)
for pid in TRAIN_IDS:
    show_patient(pid, max_slices=3)

for pid in TEST_IDS:
    show_patient(pid, max_slices=3)
```

Conclusion for above :

- Baseline scans were rigidly registered to follow-up scans.
- All images were resampled to 0.5 mm isotropic resolution, intensity-normalized, and padded to a common spatial size.
- Ground-truth lesion masks were transformed into the follow-up reference space

Step 5.2: Preprocess Test Patients

```
import os, re, glob
import numpy as np
```

```
import SimpleITK as sitk

# -------- PATHS --------
ROOTS = [
    "/content/drive/MyDrive/14 MS patients",
    "/content/drive/MyDrive/50 patients",
]
OUT_ROOT = "/content/drive/MyDrive/ms_preprocessed"
os.makedirs(OUT_ROOT, exist_ok=True)

# ✅ your TEST list (put all 7)
TEST_IDS = ["00960975", "00981157", "01003818", "01019493", "01036141", "01083481", "01135143"]

TARGET_SPACING = (0.5, 0.5, 0.5)   # (x,y,z)
TARGET_SHAPE   = (384, 512, 512)   # (x,y,z) PAD ONLY

IGNORE_IDS = {"00408990"}

# ---------------- HELPERS ----------------
def parse_date(name: str):
    m = re.search(r"(19|20)\d{6}", name)
    return int(m.group(0)) if m else None

def find_patient_dir(pid: str):
    for root in ROOTS:
        pdir = os.path.join(root, pid)
        if os.path.isdir(pdir):
            return pdir
    return None

def pick_patient_files(pid: str):
    if pid in IGNORE_IDS:
        raise FileNotFoundError(f"{pid} is ignored")

    pdir = find_patient_dir(pid)
    if pdir is None:
        raise FileNotFoundError(f"Patient folder not found in ROOTS: {pid}")

    nii = sorted(glob.glob(os.path.join(pdir, "*.nii.gz")))
    if not nii:
        raise FileNotFoundError(f"No .nii.gz files found in {pdir}")

    # label (Segmentation-...label...)
    label_candidates = [
        f for f in nii
        if os.path.basename(f).lower().startswith("segmentation") and "label" in os.path.basename(f).lower()
    ]
    if not label_candidates:
        raise FileNotFoundError(f"No segmentation label file found for {pid}")
    label = sorted(label_candidates)[0]

    # followup_registered (*_to_FLAIR1 or *_to_FLAIR_1)
    reg_candidates = [
        f for f in nii
        if re.search(r"_to_FLAIR_?1\.nii\.gz$", os.path.basename(f), flags=re.IGNORECASE)
    ]
    if not reg_candidates:
        raise FileNotFoundError(f"No registered follow-up '*_to_FLAIR1.nii.gz' or '*_to_FLAIR_1.nii.gz' for {pid}")
    follow_reg = reg_candidates[0]

    # baseline = earliest dated scan (exclude label + follow_reg)
    dated = []
    for f in nii:
        if f == label or f == follow_reg:
            continue
        d = parse_date(os.path.basename(f))
        if d is not None:
            dated.append((d, f))

    if not dated:
        raise FileNotFoundError(f"No dated scan found for baseline for {pid}")

    dated.sort(key=lambda x: x[0])
    baseline = dated[0][1]

    return pdir, baseline, follow_reg, label
```

```
def resample_to_spacing(img, out_spacing, interpolator, default_value=0.0):
    in_spacing = img.GetSpacing()
    in_size    = img.GetSize()  # (x,y,z)

    out_size = [
        int(np.round(in_size[i] * (in_spacing[i] / out_spacing[i])))
        for i in range(3)
    ]

    ref = sitk.Image(out_size, img.GetPixelID())
    ref.SetSpacing(out_spacing)
    ref.SetDirection(img.GetDirection())
    ref.SetOrigin(img.GetOrigin())

    return sitk.Resample(img, ref, sitk.Transform(), interpolator, default_value, img.GetPixelID())

def center_pad_only(img, target_size_xyz, pad_value=0.0):
    """Pad around center to target_size_xyz (x,y,z). NO CROPPING."""
    size = np.array(list(img.GetSize()), dtype=int)
    target = np.array(list(target_size_xyz), dtype=int)

    if np.any(size > target):
        raise ValueError(f"Resampled size {tuple(size)} exceeds TARGET_SHAPE {tuple(target)} (cropping would be needed).")

    pad_lower = ((target - size) // 2).astype(int).tolist()
    pad_upper = ((target - size) - np.array(pad_lower)).astype(int).tolist()

    pad = sitk.ConstantPadImageFilter()
    pad.SetPadLowerBound(pad_lower)
    pad.SetPadUpperBound(pad_upper)
    pad.SetConstant(pad_value)
    return pad.Execute(img)

def simple_foreground_mask(img):
    arr = sitk.GetArrayFromImage(img).astype(np.float32)
    nz = arr[arr > 0]
    if nz.size == 0:
        m = arr != 0
    else:
        thr = np.percentile(nz, 5)
        m = arr > thr
    out = sitk.GetImageFromArray(m.astype(np.uint8))
    out.CopyInformation(img)
    return out

def zscore_in_mask(vol, mask):
    v = sitk.GetArrayFromImage(vol).astype(np.float32)
    m = sitk.GetArrayFromImage(mask) > 0
    if m.sum() == 0:
        m = v != 0
    vals = v[m]
    mu = float(vals.mean())
    sd = float(vals.std() + 1e-6)
    v = (v - mu) / sd
    out = sitk.GetImageFromArray(v)
    out.CopyInformation(vol)
    return out

# ---------------- RUN TEST ----------------
print("TARGET_SPACING:", TARGET_SPACING)
print("TARGET_SHAPE  :", TARGET_SHAPE, "(PAD ONLY)")

for pid in TEST_IDS:
    print("\n" + "="*90)
    print("Preprocessing TEST patient:", pid)

    try:
        pdir, baseline_p, followreg_p, label_p = pick_patient_files(pid)
        print("Folder  :", pdir)
        print("Baseline:", os.path.basename(baseline_p))
        print("FollowR :", os.path.basename(followreg_p))
        print("Label   :", os.path.basename(label_p))

        b  = sitk.ReadImage(baseline_p)
        fr = sitk.ReadImage(followreg_p)
        gt = sitk.ReadImage(label_p)
```

```
            # 1) Resample to 0.5mm
            b_05  = resample_to_spacing(b,  TARGET_SPACING, sitk.sitkLinear, 0.0)
            fr_05 = resample_to_spacing(fr, TARGET_SPACING, sitk.sitkLinear, 0.0)
            gt_05 = resample_to_spacing(gt, TARGET_SPACING, sitk.sitkNearestNeighbor, 0)

                    # 2) PAD ONLY to fixed TARGET_SHAPE
            b_fix  = center_pad_only(b_05,  TARGET_SHAPE, pad_value=0.0)
            fr_fix = center_pad_only(fr_05, TARGET_SHAPE, pad_value=0.0)
            gt_fix = center_pad_only(gt_05, TARGET_SHAPE, pad_value=0)

            # 2.5) Force GT to be binary after resampling/padding
            gt_arr = sitk.GetArrayFromImage(gt_fix).astype(np.float32)
            gt_arr = (gt_arr > 0.5).astype(np.uint8)
            gt_fix_bin = sitk.GetImageFromArray(gt_arr)
            gt_fix_bin.CopyInformation(gt_fix)

            # 3) Z-score using follow-up registered foreground mask
            mask = simple_foreground_mask(fr_fix)
            b_norm  = zscore_in_mask(b_fix,  mask)
            fr_norm = zscore_in_mask(fr_fix, mask)

            # Optional debug print
            print("Postprocessed GT voxels:", int(gt_arr.sum()))

            # 4) Save (NEW filenames)
            out_dir = os.path.join(OUT_ROOT, pid)
            os.makedirs(out_dir, exist_ok=True)

            sitk.WriteImage(b_norm,     os.path.join(out_dir, "baseline_0p5_norm_padded.nii.gz"))
            sitk.WriteImage(fr_norm,    os.path.join(out_dir, "followup_registered_0p5_norm_padded.nii.gz"))
            sitk.WriteImage(gt_fix_bin, os.path.join(out_dir, "label_0p5_padded.nii.gz"))

            print("Saved to:", out_dir)
            print("Done ✅")

        except Exception as e:
            print(f"❌ Failed for {pid}: {e}")

    print("\nAll TEST patients preprocessed ✅")
```

Step 6: Patch Dataset Construction (Training Only)

build a patch dataset + dataloader from the preprocessed training patient

Input patch = (2, D, H, W) (baseline + followup as channels)

Target patch = (1, D, H, W) (new lesions)

Balanced sampling: 50% positive patches, 50% negative patches

Then we train Pipeline #1: Modified 3D U-Net

Step 6.1: Install & Import Deep Learning Libraries

```
!pip -q install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

```
import os
import numpy as np
import nibabel as nib
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
```

Step 6.2: Patch Dataset Parameters and Balanced Patch Dataset (On-the-Fly Sampling)

```
import os
import numpy as np
import nibabel as nib
import torch
from torch.utils.data import Dataset, DataLoader
```

```
# ================================
# MULTI-PATIENT PATCH TRAINING
# IMPROVED PATCH SAMPLING
# ================================

PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"

TRAIN_IDS = [
    "00713519","01092669","01386182","01406050","01722254",
    "01406050_1","01563167","01581148_2","01581148_4","01668999",
    "00516779","00749499","00832617","00890265","00890509",
    "00907490","00952337","00967329","00967939","01137872",
]

PATCH_SIZE = (64, 64, 64)
SAMPLES_PER_EPOCH = 800

# increased from 0.5
POS_FRACTION = 0.7

# require enough lesion voxels inside a positive patch
MIN_POS_VOXELS = 50

# --------- filenames ---------
B_NAME = "baseline_0p5_norm_padded.nii.gz"
F_NAME = "followup_registered_0p5_norm_padded.nii.gz"
Y_NAME = "label_0p5_padded.nii.gz"


def load_preprocessed(pid):
    pdir = os.path.join(PREP_ROOT, pid)

    b_path = os.path.join(pdir, B_NAME)
    f_path = os.path.join(pdir, F_NAME)
    y_path = os.path.join(pdir, Y_NAME)

    missing = [p for p in [b_path, f_path, y_path] if not os.path.exists(p)]
    if missing:
        raise FileNotFoundError(
            f"[{pid}] Missing preprocessed files:\n" +
            "\n".join(" - " + os.path.basename(m) for m in missing)
        )

    b = nib.load(b_path).get_fdata(dtype=np.float32)
    f = nib.load(f_path).get_fdata(dtype=np.float32)
    y = nib.load(y_path).get_fdata(dtype=np.float32)

    # force binary mask
    y = (y > 0.5).astype(np.uint8)

    if b.shape != f.shape or b.shape != y.shape:
        raise ValueError(f"[{pid}] Shape mismatch: b={b.shape}, f={f.shape}, y={y.shape}")

    return b, f, y


def valid_patch_bounds(shape, patch):
    D, H, W = shape
    pd, ph, pw = patch
    if D < pd or H < ph or W < pw:
        raise ValueError(f"Patch {patch} bigger than volume shape {shape}")
    return (D - pd, H - ph, W - pw)


class MultiPatientBalancedPatchDataset(Dataset):
    """
    Each __getitem__:
      1) randomly selects a patient
      2) samples a positive or negative patch from that patient
      3) positive patches are encouraged to contain real lesion content
    """
    def __init__(
        self,
        patient_ids,
        patch_size=(64, 64, 64),
        samples_per_epoch=400,
```

```
        pos_fraction=0.7,
        min_pos_voxels=50,
        debug=False
    ):
        self.pids = list(patient_ids)
        self.patch = patch_size
        self.N = int(samples_per_epoch)
        self.pos_fraction = float(pos_fraction)
        self.min_pos_voxels = int(min_pos_voxels)
        self.debug = debug

        # load all patients into memory
        self.data = {}

        print("Loaded patients into dataset:")
        for pid in self.pids:
            b, f, y = load_preprocessed(pid)

            pos_coords = np.argwhere(y > 0)
            has_pos = pos_coords.shape[0] > 0
            maxz, maxy, maxx = valid_patch_bounds(b.shape, self.patch)

            self.data[pid] = {
                "b": b,
                "f": f,
                "y": y,
                "pos_coords": pos_coords,
                "has_pos": has_pos,
                "maxz": maxz,
                "maxy": maxy,
                "maxx": maxx,
                "shape": b.shape,
                "gt_voxels": int(y.sum())
            }

            print(
                f" - {pid}: shape={b.shape}, "
                f"GT_voxels={int(y.sum())}, "
                f"has_pos={has_pos}"
            )

    def __len__(self):
        return self.N

    def _random_start(self, maxz, maxy, maxx):
        z = np.random.randint(0, maxz + 1)
        y0 = np.random.randint(0, maxy + 1)
        x = np.random.randint(0, maxx + 1)
        return z, y0, x

    def _pos_start_centered(self, pos_coords, maxz, maxy, maxx):
        cz, cy, cx = pos_coords[np.random.randint(0, len(pos_coords))]
        pd, ph, pw = self.patch

        z = int(np.clip(cz - pd // 2, 0, maxz))
        y0 = int(np.clip(cy - ph // 2, 0, maxy))
        x = int(np.clip(cx - pw // 2, 0, maxx))
        return z, y0, x

    def _extract_patch(self, d, z, y0, x):
        pd, ph, pw = self.patch
        b_patch = d["b"][z:z+pd, y0:y0+ph, x:x+pw]
        f_patch = d["f"][z:z+pd, y0:y0+ph, x:x+pw]
        y_patch = d["y"][z:z+pd, y0:y0+ph, x:x+pw]
        return b_patch, f_patch, y_patch

    def _sample_positive_patch(self, d, max_tries=30):
        """
        Try multiple lesion-centered crops until we get enough lesion voxels.
        """
        for _ in range(max_tries):
            z, y0, x = self._pos_start_centered(
                d["pos_coords"], d["maxz"], d["maxy"], d["maxx"]
            )
            _, _, y_patch = self._extract_patch(d, z, y0, x)

            if int(y_patch.sum()) >= self.min_pos_voxels:
```

```
                return z, y0, x

        # fallback
        return self._pos_start_centered(
            d["pos_coords"], d["maxz"], d["maxy"], d["maxx"]
        )

    def _sample_negative_patch(self, d, max_tries=20):
        """
        Try to find a patch with no lesion voxels.
        """
        for _ in range(max_tries):
            z, y0, x = self._random_start(d["maxz"], d["maxy"], d["maxx"])
            _, _, y_patch = self._extract_patch(d, z, y0, x)

            if int(y_patch.sum()) == 0:
                return z, y0, x

        # fallback
        return self._random_start(d["maxz"], d["maxy"], d["maxx"])

    def __getitem__(self, idx):
        pid = self.pids[np.random.randint(0, len(self.pids))]
        d = self.data[pid]

        want_pos = (np.random.rand() < self.pos_fraction) and d["has_pos"]

        if want_pos:
            z, y0, x = self._sample_positive_patch(d)
            patch_type = "POS"
        else:
            z, y0, x = self._sample_negative_patch(d)
            patch_type = "NEG"

        b_patch, f_patch, y_patch = self._extract_patch(d, z, y0, x)

        X = np.stack([b_patch, f_patch], axis=0).astype(np.float32)   # (2, D, H, W)
        Y = y_patch[None, ...].astype(np.float32)                     # (1, D, H, W)

        if self.debug and idx < 10:
            print(
                f"[DEBUG] idx={idx} pid={pid} type={patch_type} "
                f"patch_GT_voxels={int(y_patch.sum())}"
            )

        return torch.from_numpy(X), torch.from_numpy(Y)


# ==============================
# Create dataset + loader
# ==============================
train_ds = MultiPatientBalancedPatchDataset(
    TRAIN_IDS,
    patch_size=PATCH_SIZE,
    samples_per_epoch=SAMPLES_PER_EPOCH,
    pos_fraction=POS_FRACTION,
    min_pos_voxels=MIN_POS_VOXELS,
    debug=True
)

train_loader = DataLoader(
    train_ds,
    batch_size=2,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

# ==============================
# Sanity check
# ==============================
Xb, Yb = next(iter(train_loader))
print("Example batch shapes:", Xb.shape, Yb.shape)
print("Input dtype:", Xb.dtype)
print("Label dtype:", Yb.dtype)
print("Label unique values:", torch.unique(Yb))
print("Lesion voxels per sample in batch:", Yb.sum(dim=(1,2,3,4)))
```

```
Loaded patients into dataset:
 - 00713519: shape=(384, 512, 512), GT_voxels=7956, has_pos=True
 - 01092669: shape=(384, 512, 512), GT_voxels=3458, has_pos=True
 - 01386182: shape=(384, 512, 512), GT_voxels=32317, has_pos=True
 - 01406050: shape=(384, 512, 512), GT_voxels=89480, has_pos=True
 - 01722254: shape=(384, 512, 512), GT_voxels=2756, has_pos=True
 - 01406050_1: shape=(384, 512, 512), GT_voxels=26006, has_pos=True
 - 01563167: shape=(384, 512, 512), GT_voxels=4282, has_pos=True
 - 01581148_2: shape=(384, 512, 512), GT_voxels=4889, has_pos=True
 - 01581148_4: shape=(384, 512, 512), GT_voxels=2664, has_pos=True
 - 01668999: shape=(384, 512, 512), GT_voxels=646, has_pos=True
 - 00516779: shape=(384, 512, 512), GT_voxels=644, has_pos=True
 - 00749499: shape=(384, 512, 512), GT_voxels=628, has_pos=True
 - 00832617: shape=(384, 512, 512), GT_voxels=20686, has_pos=True
 - 00890265: shape=(384, 512, 512), GT_voxels=286, has_pos=True
 - 00890509: shape=(384, 512, 512), GT_voxels=832, has_pos=True
 - 00907490: shape=(384, 512, 512), GT_voxels=596, has_pos=True
 - 00952337: shape=(384, 512, 512), GT_voxels=800, has_pos=True
 - 00967329: shape=(384, 512, 512), GT_voxels=308, has_pos=True
 - 00967939: shape=(384, 512, 512), GT_voxels=1408, has_pos=True
 - 01137872: shape=(384, 512, 512), GT_voxels=2798, has_pos=True
Example batch shapes: torch.Size([2, 2, 64, 64, 64]) torch.Size([2, 1, 64, 64, 64])
Input dtype: torch.float32
Label dtype: torch.float32
Label unique values: tensor([0., 1.])
Lesion voxels per sample in batch: tensor([9497.,    0.])
```

```
import os

root = "/content/drive/MyDrive/ms_preprocessed"
print("Root exists:", os.path.exists(root))
print("Top-level entries:", os.listdir(root)[:20])
```

```
Root exists: True
Top-level entries: ['_checkpoints']
```

Step 7: Model and Training (NeuroPoly Pipeline #1)

Step 7.1: Model Architecture — Modified 3D U-Net (Pipeline #1)

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv1 = nn.Conv3d(in_ch, out_ch, 3, padding=1)
        self.in1   = nn.InstanceNorm3d(out_ch)
        self.conv2 = nn.Conv3d(out_ch, out_ch, 3, padding=1)
        self.in2   = nn.InstanceNorm3d(out_ch)
        self.act   = nn.LeakyReLU(0.01, inplace=True)

    def forward(self, x):
        x = self.act(self.in1(self.conv1(x)))
        x = self.act(self.in2(self.conv2(x)))
        return x

class Down(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.down = nn.Conv3d(in_ch, out_ch, kernel_size=3, stride=2, padding=1)  # strided conv downsample
        self.in0  = nn.InstanceNorm3d(out_ch)
        self.act  = nn.LeakyReLU(0.01, inplace=True)
        self.block = ConvBlock(out_ch, out_ch)

    def forward(self, x):
        x = self.act(self.in0(self.down(x)))
        x = self.block(x)
        return x

class Up(nn.Module):
    def __init__(self, in_ch, skip_ch, out_ch):
        super().__init__()
        self.up = nn.Upsample(scale_factor=2, mode="nearest")
        self.conv1x1 = nn.Conv3d(in_ch, out_ch, kernel_size=1)      # match dims after up
        self.skip1x1 = nn.Conv3d(skip_ch, out_ch, kernel_size=1)    # match skip dims
        self.block = ConvBlock(out_ch*2, out_ch)

    def forward(self, x, skip):
```

```
        x = self.up(x)
        x = self.conv1x1(x)
        skip = self.skip1x1(skip)
        x = torch.cat([skip, x], dim=1)
        return self.block(x)

class ModifiedUNet3D(nn.Module):
    def __init__(self, in_ch=2, base=16):
        super().__init__()
        self.enc0 = ConvBlock(in_ch, base)
        self.enc1 = Down(base, base*2)
        self.enc2 = Down(base*2, base*4)
        self.enc3 = Down(base*4, base*8)

        self.bottleneck = Down(base*8, base*16)

        self.dec3 = Up(base*16, base*8, base*8)
        self.dec2 = Up(base*8,  base*4, base*4)
        self.dec1 = Up(base*4,  base*2, base*2)
        self.dec0 = Up(base*2,  base,   base)

        self.out = nn.Conv3d(base, 1, kernel_size=1)

    def forward(self, x):
        s0 = self.enc0(x)
        s1 = self.enc1(s0)
        s2 = self.enc2(s1)
        s3 = self.enc3(s2)
        b  = self.bottleneck(s3)
        x  = self.dec3(b, s3)
        x  = self.dec2(x, s2)
        x  = self.dec1(x, s1)
        x  = self.dec0(x, s0)
        return self.out(x)  # logits
```

Step 7.2-A: Loss Function and Evaluation Metric

```
import os
import numpy as np
import nibabel as nib
import torch
import torch.nn.functional as F

# ============================================================
# CONFIG
# ============================================================
PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"

TRAIN_IDS = [
    "00713519","01092669","01386182","01406050","01722254",
    "01406050_1","01563167","01581148_2","01581148_4","01668999",
    "00516779","00749499","00832617","00890265","00890509",
    "00907490","00952337","00967329","00967939","01137872",
]

TEST_IDS = [
    "00960975","00981157","01003818","01019493",
    "01036141","01083481","01135143"
]

PATCH_SIZE = (64, 64, 64)
STRIDE     = (32, 32, 32)
THR        = 0.5

MAX_EPOCHS  = 500
EVAL_EPOCHS = set(range(1, MAX_EPOCHS + 1))

RUN_NAME = "train20_ep500_lr3e-4_wd1e-6_dicebce"
CKPT_DIR = f"/content/drive/MyDrive/ms_preprocessed/_checkpoints/{RUN_NAME}"
os.makedirs(CKPT_DIR, exist_ok=True)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
# ============================================================
# LOSSES / METRICS
# ============================================================
def soft_dice_loss(logits, targets, eps=1e-6):
    probs = torch.sigmoid(logits)
    num = 2 * (probs * targets).sum(dim=(1, 2, 3, 4))
    den = probs.sum(dim=(1, 2, 3, 4)) + targets.sum(dim=(1, 2, 3, 4)) + eps
    dice = num / den
    return 1 - dice.mean()

def dice_bce_loss(logits, targets, dice_weight=0.5, bce_weight=0.5):
    dice = soft_dice_loss(logits, targets)
    bce = F.binary_cross_entropy_with_logits(logits, targets)
    return dice_weight * dice + bce_weight * bce

def dice_score(pred, gt, eps=1e-8):
    pred = pred.astype(bool)
    gt   = gt.astype(bool)
    inter = np.logical_and(pred, gt).sum()
    denom = pred.sum() + gt.sum()
    return (2.0 * inter + eps) / (denom + eps)

# ============================================================
# LOADING
# ============================================================
def _find_one(pdir, candidates):
    for name in candidates:
        p = os.path.join(pdir, name)
        if os.path.exists(p):
            return p
    return None

def load_preprocessed(pid):
    pdir = os.path.join(PREP_ROOT, pid)
    if not os.path.isdir(pdir):
        raise FileNotFoundError(f"Missing preprocessed folder: {pdir}")

    b_path = _find_one(pdir, [
        "baseline_0p5_norm_padded.nii.gz",
        "baseline_0p5_registered_to_followup_norm.nii.gz",
    ])
    f_path = _find_one(pdir, [
        "followup_registered_0p5_norm_padded.nii.gz",
        "followup_0p5_norm.nii.gz",
    ])
    y_path = _find_one(pdir, [
        "label_0p5_padded.nii.gz",
        "label_0p5_in_followup_space.nii.gz",
    ])

    if b_path is None or f_path is None or y_path is None:
        raise FileNotFoundError(
            f"{pid}: missing one of files. Found:\n"
            f"  b={b_path}\n  f={f_path}\n  y={y_path}"
        )

    b = nib.load(b_path).get_fdata().astype(np.float32)
    f = nib.load(f_path).get_fdata().astype(np.float32)
    y = nib.load(y_path).get_fdata().astype(np.float32)
    y = (y > 0.5).astype(np.uint8)

    if b.shape != f.shape or b.shape != y.shape:
        raise ValueError(f"{pid}: shape mismatch b={b.shape}, f={f.shape}, y={y.shape}")

    return b, f, y

# ============================================================
# SLIDING WINDOW
# ============================================================
def sliding_window_predict(model, b, f, patch_size=(64,64,64), stride=(32,32,32)):
    model.eval()

    D, H, W = b.shape
    pd, ph, pw = patch_size
    sd, sh, sw = stride

    prob  = np.zeros((D, H, W), dtype=np.float32)
```

```
    count = np.zeros((D, H, W), dtype=np.float32)

    z_starts = list(range(0, max(D - pd + 1, 1), sd))
    y_starts = list(range(0, max(H - ph + 1, 1), sh))
    x_starts = list(range(0, max(W - pw + 1, 1), sw))

    if z_starts[-1] != D - pd:
        z_starts.append(D - pd)
    if y_starts[-1] != H - ph:
        y_starts.append(H - ph)
    if x_starts[-1] != W - pw:
        x_starts.append(W - pw)

    with torch.no_grad():
        for z in z_starts:
            for y0 in y_starts:
                for x in x_starts:
                    b_patch = b[z:z+pd, y0:y0+ph, x:x+pw]
                    f_patch = f[z:z+pd, y0:y0+ph, x:x+pw]

                    X = np.stack([b_patch, f_patch], axis=0)[None]
                    X = torch.from_numpy(X).to(device)

                    logits = model(X)
                    p = torch.sigmoid(logits)[0, 0].detach().cpu().numpy()

                    prob[z:z+pd, y0:y0+ph, x:x+pw]  += p
                    count[z:z+pd, y0:y0+ph, x:x+pw] += 1.0

    count[count == 0] = 1.0
    return prob / count

def evaluate_on_tests(model, thr=0.5):
    model.eval()
    results = {}

    with torch.no_grad():
        for pid in TEST_IDS:
            b, f, gt = load_preprocessed(pid)

            prob = sliding_window_predict(
                model,
                b,
                f,
                patch_size=PATCH_SIZE,
                stride=STRIDE
            )

            pred = (prob >= thr).astype(np.uint8)
            results[pid] = dice_score(pred, gt)

    model.train()
    return results

# ============================================================
# MODEL / OPTIMIZER
# ============================================================
# Make sure ModifiedUNet3D is already defined above this cell
model = ModifiedUNet3D(in_ch=2, base=16).to(device)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-6)

# ============================================================
# CHECKPOINT HELPERS
# ============================================================
def save_ckpt(epoch, model, opt, path, results=None, avg_loss=None, mean_dice=None):
    torch.save({
        "epoch": epoch,
        "model": model.state_dict(),
        "opt": opt.state_dict(),
        "results": results,
        "avg_loss": avg_loss,
        "mean_dice": mean_dice,
    }, path)

def load_ckpt(path, model, opt):
    ckpt = torch.load(path, map_location=device, weights_only=False)
    model.load_state_dict(ckpt["model"])
```

```
    opt.load_state_dict(ckpt["opt"])
    return int(ckpt.get("epoch", 0))

# ============================================================
# TRAIN LOOP
# ============================================================
def train(model, opt, train_loader, start_epoch=1, max_epochs=500):
    model.train()
    history = []

    for epoch in range(start_epoch, max_epochs + 1):
        running = 0.0
        n = 0

        for X, Y in train_loader:
            X = X.to(device, non_blocking=True)
            Y = Y.to(device, non_blocking=True)

            opt.zero_grad(set_to_none=True)
            logits = model(X)
            loss = dice_bce_loss(logits, Y)
            loss.backward()
            opt.step()

            running += float(loss.item())
            n += 1

        avg_loss = running / max(n, 1)
        print(f"Epoch {epoch:03d}/{max_epochs} | train_loss={avg_loss:.4f}")

        if epoch in EVAL_EPOCHS:
            results = evaluate_on_tests(model, thr=THR)
            mean_dice = float(np.mean(list(results.values()))) if results else 0.0

            print("  TEST Dice:", {k: round(v, 4) for k, v in results.items()})
            print("  TEST Mean:", round(mean_dice, 4))

            ckpt_path = os.path.join(CKPT_DIR, f"epoch_{epoch:03d}.pt")
            save_ckpt(
                epoch=epoch,
                model=model,
                opt=opt,
                path=ckpt_path,
                results=results,
                avg_loss=avg_loss,
                mean_dice=mean_dice
            )
            print("  Saved:", ckpt_path)

            history.append({
                "epoch": epoch,
                "train_loss": avg_loss,
                "test_results": results,
                "mean_dice": mean_dice
            })

    return history

print("✅ Setup complete.")
print("Next run:")
print("history = train(model, opt, train_loader, start_epoch=1, max_epochs=MAX_EPOCHS)")
```

```
Device: cuda
✅ Setup complete.
Next run:
history = train(model, opt, train_loader, start_epoch=1, max_epochs=MAX_EPOCHS)
```

```
history = train(model, opt, train_loader, start_epoch=1, max_epochs=MAX_EPOCHS)
```

```
[DEBUG] idx=9 pid=01137872 type=NEG patch_GT_voxels=0
[DEBUG] idx=0 pid=00749499 type=POS patch_GT_voxels=628
[DEBUG] idx=5 pid=01406050_1 type=POS patch_GT_voxels=9450
[DEBUG] idx=8 pid=01406050_1 type=POS patch_GT_voxels=9710
[DEBUG] idx=1 pid=00907490 type=POS patch_GT_voxels=352
[DEBUG] idx=3 pid=00713519 type=POS patch_GT_voxels=756
[DEBUG] idx=4 pid=01386182 type=NEG patch_GT_voxels=0
[DEBUG] idx=2 pid=00749499 type=POS patch_GT_voxels=628
[DEBUG] idx=6 pid=01581148_2 type=NEG patch_GT_voxels=0
```

```
[DEBUG] idx=7 pid=01137872 type=POS patch_GT_voxels=976
Epoch 001/500 | train_loss=0.7161
  TEST Dice: {'00960975': np.float64(0.0112), '00981157': np.float64(0.1119), '01003818': np.float64(0.0048), '01019493': np.f
  TEST Mean: 0.0215
  Saved: /content/drive/MyDrive/ms_preprocessed/_checkpoints/train20_ep500_lr3e-4_wd1e-6_dicebce/epoch_001.pt
[DEBUG] idx=6 pid=00749499 type=NEG patch_GT_voxels=0
[DEBUG] idx=7 pid=01406050_1 type=POS patch_GT_voxels=3496
[DEBUG] idx=2 pid=01137872 type=POS patch_GT_voxels=976
[DEBUG] idx=4 pid=00967329 type=POS patch_GT_voxels=248
[DEBUG] idx=8 pid=00890265 type=POS patch_GT_voxels=204
[DEBUG] idx=5 pid=00967939 type=POS patch_GT_voxels=776
[DEBUG] idx=3 pid=00967939 type=NEG patch_GT_voxels=0
[DEBUG] idx=1 pid=01581148_2 type=POS patch_GT_voxels=1104
[DEBUG] idx=0 pid=00952337 type=NEG patch_GT_voxels=0
[DEBUG] idx=9 pid=01386182 type=NEG patch_GT_voxels=0
Epoch 002/500 | train_loss=0.6425
  TEST Dice: {'00960975': np.float64(0.0185), '00981157': np.float64(0.1851), '01003818': np.float64(0.0108), '01019493': np.f
  TEST Mean: 0.0363
  Saved: /content/drive/MyDrive/ms_preprocessed/_checkpoints/train20_ep500_lr3e-4_wd1e-6_dicebce/epoch_002.pt
[DEBUG] idx=4 pid=00713519 type=POS patch_GT_voxels=1866
[DEBUG] idx=7 pid=01137872 type=NEG patch_GT_voxels=0
[DEBUG] idx=8 pid=01581148_4 type=POS patch_GT_voxels=792
[DEBUG] idx=9 pid=00890509 type=POS patch_GT_voxels=252
[DEBUG] idx=0 pid=01137872 type=POS patch_GT_voxels=1822
[DEBUG] idx=6 pid=01722254 type=POS patch_GT_voxels=904
[DEBUG] idx=2 pid=00749499 type=NEG patch_GT_voxels=0
[DEBUG] idx=5 pid=00967329 type=POS patch_GT_voxels=248
[DEBUG] idx=1 pid=00952337 type=POS patch_GT_voxels=666
[DEBUG] idx=3 pid=00713519 type=NEG patch_GT_voxels=0
Epoch 003/500 | train_loss=0.5834
  TEST Dice: {'00960975': np.float64(0.1114), '00981157': np.float64(0.529), '01003818': np.float64(0.0434), '01019493': np.flo
  TEST Mean: 0.1248
  Saved: /content/drive/MyDrive/ms_preprocessed/_checkpoints/train20_ep500_lr3e-4_wd1e-6_dicebce/epoch_003.pt
[DEBUG] idx=9 pid=01092669 type=POS patch_GT_voxels=2334
[DEBUG] idx=2 pid=01137872 type=NEG patch_GT_voxels=0
[DEBUG] idx=1 pid=01581148_2 type=POS patch_GT_voxels=1279
[DEBUG] idx=6 pid=00749499 type=POS patch_GT_voxels=628
[DEBUG] idx=3 pid=01563167 type=POS patch_GT_voxels=2918
[DEBUG] idx=4 pid=01581148_4 type=NEG patch_GT_voxels=0
[DEBUG] idx=7 pid=01092669 type=POS patch_GT_voxels=2334
[DEBUG] idx=8 pid=00890265 type=NEG patch_GT_voxels=0
[DEBUG] idx=5 pid=01581148_2 type=POS patch_GT_voxels=2498
[DEBUG] idx=0 pid=01406050 type=POS patch_GT_voxels=9292
Epoch 004/500 | train_loss=0.5390
  TEST Dice: {'00960975': np.float64(0.0138), '00981157': np.float64(0.1409), '01003818': np.float64(0.0072), '01019493': np.f
  TEST Mean: 0.0275
  Saved: /content/drive/MyDrive/ms_preprocessed/_checkpoints/train20_ep500_lr3e-4_wd1e-6_dicebce/epoch_004.pt
[DEBUG] idx=7 pid=00749499 type=POS patch_GT_voxels=628
```

Step 7.3: Visual Evaluation — Ground Truth vs Prediction

```
TEST_IDS = ["00960975", "00981157"]
THR = 0.5  # try 0.3 too

model.eval()
results = {}

for pid in TEST_IDS:
    b, f, gt = load_preprocessed(pid)  # gt is binary (0/1)
    prob = sliding_window_predict(b, f, patch_size=PATCH_SIZE, stride=STRIDE)
    pred = (prob >= THR).astype(np.uint8)

    d = dice_score(pred, gt)
    results[pid] = float(d)
    print(f"Patient {pid} Dice @ thr={THR}: {d:.4f}")

avg_dice = sum(results.values()) / len(results)
print(f"\nAverage test Dice @ thr={THR}: {avg_dice:.4f}")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_2136/3879095124.py in <cell line: 0>()
      7 for pid in TEST_IDS:
      8     b, f, gt = load_preprocessed(pid)  # gt is binary (0/1)
----> 9     prob = sliding_window_predict(b, f, patch_size=PATCH_SIZE, stride=STRIDE)
     10     pred = (prob >= THR).astype(np.uint8)
     11

TypeError: sliding_window_predict() missing 1 required positional argument: 'f'
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import os

ROOT = "/content/drive/MyDrive/ms_preprocessed/_checkpoints"
print(os.listdir(ROOT))
```

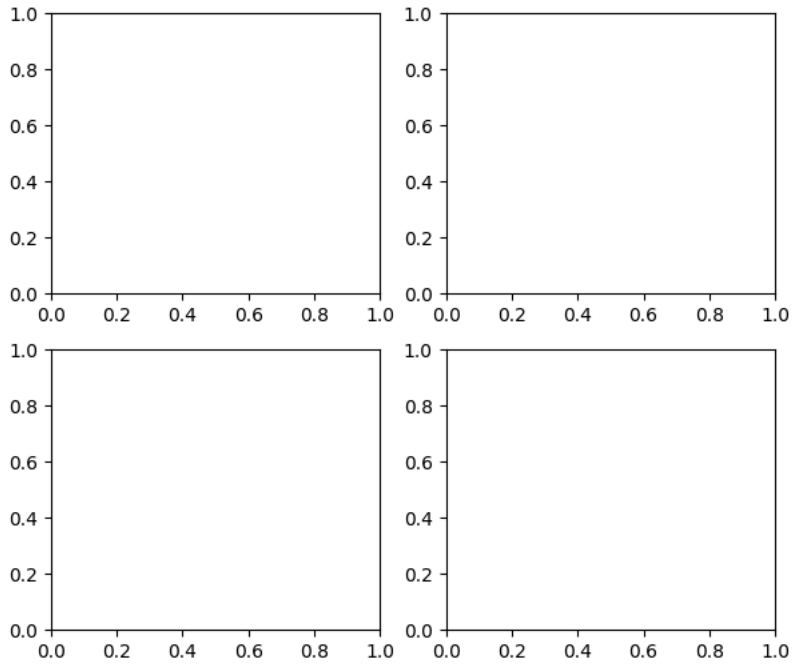
```
['unet_epoch200.pt', 'unet_epoch25.pt', 'unet_epoch50.pt', 'unet_epoch75.pt', 'unet_epoch100.pt', 'train10_ep100_lr1e-4', 'train
```

```
PIDS = ["00981157", "01019493", "01135143"]
THR = 0.5
```

```
import os
import numpy as np
import torch
import matplotlib.pyplot as plt

# ============================================================
# CONFIG
# ============================================================
RUN_NAME = "train20_ep500_lr3e-4_wd1e-6_dicebce"
CKPT_DIR = f"/content/drive/MyDrive/ms_preprocessed/_checkpoints/{RUN_NAME}"

EPOCHS_TO_VIEW = [100, 200, 300, 336]   # change as you want
TEST_PATIENTS = ["00960975", "00981157"]   # choose 2 or 3 test patients
THR = 0.5

OUT_VIS_DIR = f"/content/drive/MyDrive/ms_preprocessed/_visual_results_{RUN_NAME}"
os.makedirs(OUT_VIS_DIR, exist_ok=True)

# ============================================================
# REQUIRED EXISTING OBJECTS
# You must already have these defined before running:
# - device
# - model class: ModifiedUNet3D
# - load_preprocessed(pid)
# - sliding_window_predict(model, b, f, patch_size, stride)
# - dice_score(pred, gt)
# ============================================================

PATCH_SIZE = (64, 64, 64)
STRIDE = (32, 32, 32)

# ============================================================
# BUILD MODEL
# ============================================================
model = ModifiedUNet3D(in_ch=2, base=16).to(device)

# ============================================================
# HELPER: choose best slice
# ============================================================
def best_slice_from_gt(gt, pred=None):
    z_gt = gt.sum(axis=(1, 2))
    if z_gt.max() > 0:
        return int(np.argmax(z_gt))

    if pred is not None:
        z_pr = pred.sum(axis=(1, 2))
        if z_pr.max() > 0:
```

```
            return int(np.argmax(z_pr))

    return gt.shape[0] // 2

# ============================================================
# PLOT ONE EPOCH
# ============================================================
def plot_epoch(epoch, thr=0.5):
    ckpt_path = os.path.join(CKPT_DIR, f"epoch_{epoch:03d}.pt")

    if not os.path.exists(ckpt_path):
        print(f"Checkpoint not found for epoch {epoch}: {ckpt_path}")
        return

    ckpt = torch.load(ckpt_path, map_location=device, weights_only=False)
    model.load_state_dict(ckpt["model"])
    model.eval()

    print("Loaded:", ckpt_path)

    fig, axes = plt.subplots(len(TEST_PATIENTS), 2, figsize=(7, 3 * len(TEST_PATIENTS)))

    if len(TEST_PATIENTS) == 1:
        axes = np.expand_dims(axes, axis=0)

    for i, pid in enumerate(TEST_PATIENTS):
        b, f, gt = load_preprocessed(pid)

        prob = sliding_window_predict(
            model, b, f,
            patch_size=PATCH_SIZE,
            stride=STRIDE
        )

        pred = (prob >= thr).astype(np.uint8)
        d = dice_score(pred, gt)

        sl = best_slice_from_gt(gt, pred)

        axes[i, 0].imshow(gt[sl], cmap="gray")
        axes[i, 0].set_title(f"Ground Truth\nPatient {pid}")
        axes[i, 0].axis("off")

        axes[i, 1].imshow(pred[sl], cmap="gray")
        axes[i, 1].set_title(f"Prediction\nPatient {pid}\nDice={d:.4f}")
        axes[i, 1].axis("off")

    plt.suptitle(f"GT vs Prediction — Epoch {epoch} (thr={thr})", fontsize=16)
    plt.tight_layout(rect=[0, 0, 1, 0.95])

    save_path = os.path.join(OUT_VIS_DIR, f"epoch_{epoch:03d}_thr{thr}.png")
    plt.savefig(save_path, dpi=200, bbox_inches="tight")
    plt.show()

    print("Saved:", save_path)

# ============================================================
# LOOP OVER EPOCHS
# ============================================================
for ep in EPOCHS_TO_VIEW:
    plot_epoch(ep, thr=THR)
```

```
Loaded: /content/drive/MyDrive/ms_preprocessed/_checkpoints/train20_ep500_lr3e-4_wd1e-6_dicebce/epoch_100.pt
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_2136/1513256177.py in <cell line: 0>()
    106 # ============================================================
    107 for ep in EPOCHS_TO_VIEW:
--> 108     plot_epoch(ep, thr=THR)

 2 frames 
/usr/local/lib/python3.12/dist-packages/numpy/_core/shape_base.py in stack(arrays, axis, out, dtype, casting)
    453     sl = (slice(None),) * axis + (_nx.newaxis,)
    454     expanded_arrays = [arr[sl] for arr in arrays]
--> 455     return _nx.concatenate(expanded_arrays, axis=axis, out=out,
    456                            dtype=dtype, casting=casting)
    457

KeyboardInterrupt:
```



PostProcessing

Imports

```
import os
import numpy as np
import pandas as pd
import nibabel as nib
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
```

Config for testing on epoch 336 only

```
# ============================================================
# CONFIG
# ============================================================
PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"
RUN_NAME = "train20_ep500_lr3e-4_wd1e-6_dicebce"
CKPT_DIR = f"/content/drive/MyDrive/ms_preprocessed/_checkpoints/{RUN_NAME}"
CKPT_PATH = os.path.join(CKPT_DIR, "epoch_336.pt")

THR = 0.5
PATCH_SIZE = (64, 64, 64)
STRIDE = (32, 32, 32)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
print("Checkpoint exists:", os.path.exists(CKPT_PATH))
print("Checkpoint path:", CKPT_PATH)

OUT_DIR = f"/content/drive/MyDrive/ms_preprocessed/_slice_analysis_{RUN_NAME}"
os.makedirs(OUT_DIR, exist_ok=True)
os.makedirs(os.path.join(OUT_DIR, "figures"), exist_ok=True)
```

```
Device: cuda
Checkpoint exists: True
Checkpoint path: /content/drive/MyDrive/ms_preprocessed/_checkpoints/train20_ep500_lr3e-4_wd1e-6_dicebce/epoch_336.pt
```

Load model once

```
model = ModifiedUNet3D(in_ch=2, base=16).to(device)

ckpt = torch.load(CKPT_PATH, map_location=device, weights_only=False)
model.load_state_dict(ckpt["model"])
model.eval()

print("Loaded checkpoint successfully.")
```

```
Loaded checkpoint successfully.
```

quick check for 2–3 patients first

```
PIDS = ["00981157", "01019493", "01135143"]   # change if needed
```

helper functions for slice analysis

```
def normalize_for_display(img2d):
    img2d = img2d.astype(np.float32)
    mn, mx = img2d.min(), img2d.max()
    return (img2d - mn) / (mx - mn + 1e-8)

def get_lesion_slices(mask):
    # assumes slices are along axis 0
    return np.where(mask.sum(axis=(1, 2)) > 0)[0]

def analyze_patient(pid, save_figures=True, max_fig_slices=None):
    print(f"\n=== Patient {pid} ===")

    b, f, gt = load_preprocessed(pid)

    prob = sliding_window_predict(
        model, b, f,
        patch_size=PATCH_SIZE,
        stride=STRIDE
    )

    pred = (prob >= THR).astype(np.uint8)
    dsc = dice_score(pred, gt)

    gt_slices = get_lesion_slices(gt)
    pred_slices = get_lesion_slices(pred)

    print("Dice:", round(dsc, 4))
    print("GT lesion slices:", gt_slices.tolist()[:20], "..." if len(gt_slices) > 20 else "")
    print("Pred lesion slices:", pred_slices.tolist()[:20], "..." if len(pred_slices) > 20 else "")

    rows = []
    all_slices = np.where((gt.sum(axis=(1,2)) > 0) | (pred.sum(axis=(1,2)) > 0))[0]

    for z in all_slices:
        gt_area = int(gt[z].sum())
        pred_area = int(pred[z].sum())

        rows.append({
            "patient_id": pid,
            "slice_index": int(z),
            "gt_has_lesion": int(gt_area > 0),
            "pred_has_lesion": int(pred_area > 0),
            "gt_area_pixels": gt_area,
            "pred_area_pixels": pred_area
```

```
            })

        df = pd.DataFrame(rows)

        # Save CSV per patient
        csv_path = os.path.join(OUT_DIR, f"{pid}_slice_summary.csv")
        df.to_csv(csv_path, index=False)

        # Save visualizations
        if save_figures:
            z_list = all_slices.copy()
            if max_fig_slices is not None:
                z_list = z_list[:max_fig_slices]

            for z in z_list:
                mri = normalize_for_display(f[z])

                overlay = np.stack([mri, mri, mri], axis=-1)
                overlay[..., 1] = np.maximum(overlay[..., 1], gt[z].astype(np.float32))   # GT green
                overlay[..., 0] = np.maximum(overlay[..., 0], pred[z].astype(np.float32)) # Pred red

                fig, axes = plt.subplots(1, 4, figsize=(16, 4))

                axes[0].imshow(mri, cmap="gray")
                axes[0].set_title(f"MRI\n{pid} | z={z}")
                axes[0].axis("off")

                axes[1].imshow(gt[z], cmap="gray")
                axes[1].set_title(f"GT\narea={int(gt[z].sum())}")
                axes[1].axis("off")

                axes[2].imshow(pred[z], cmap="gray")
                axes[2].set_title(f"Pred\narea={int(pred[z].sum())}")
                axes[2].axis("off")

                axes[3].imshow(overlay)
                axes[3].set_title(f"Overlay\nDice={dsc:.4f}")
                axes[3].axis("off")

                plt.tight_layout()

                fig_path = os.path.join(OUT_DIR, "figures", f"{pid}_slice_{z:03d}.png")
                plt.savefig(fig_path, dpi=180, bbox_inches="tight")
                plt.close()

        return {
            "patient_id": pid,
            "dice": float(dsc),
            "n_gt_slices": int(len(gt_slices)),
            "n_pred_slices": int(len(pred_slices)),
            "gt_slice_list": gt_slices.tolist(),
            "pred_slice_list": pred_slices.tolist(),
            "csv_path": csv_path
        }, df
```

test on 2–3 patients first

```
    all_patient_stats = []
    all_slice_tables = []

    for pid in PIDS:
        stats, df = analyze_patient(pid, save_figures=True, max_fig_slices=20)
        all_patient_stats.append(stats)
        all_slice_tables.append(df)

    patient_stats_df = pd.DataFrame(all_patient_stats)
    patient_stats_df
```

```
=== Patient 00981157 ===
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_2136/934332359.py in <cell line: 0>()
      3
      4 for pid in PIDS:
----> 5     stats, df = analyze_patient(pid, save_figures=True, max_fig_slices=20)
      6     all_patient_stats.append(stats)
      7     all_slice_tables.append(df)

 1 frames
/tmp/ipykernel_2136/921268099.py in sliding_window_predict(model, b, f, patch_size, stride)
    137
    138                     logits = model(X)
--> 139                     p = torch.sigmoid(logits)[0, 0].detach().cpu().numpy()
    140
    141                     prob[z:z+pd, y0:y0+ph, x:x+pw]  += p

KeyboardInterrupt:
```

save overall summary for the 2–3 patients

```
patient_stats_path = os.path.join(OUT_DIR, "patient_level_summary_small_test.csv")
patient_stats_df.to_csv(patient_stats_path, index=False)
print("Saved:", patient_stats_path)

all_slices_df = pd.concat(all_slice_tables, ignore_index=True)
all_slices_path = os.path.join(OUT_DIR, "slice_level_summary_small_test.csv")
all_slices_df.to_csv(all_slices_path, index=False)
print("Saved:", all_slices_path)
```

```
Saved: /content/drive/MyDrive/ms_preprocessed/_slice_analysis_train20_ep500_lr3e-4_wd1e-6_dicebce/patient_level_summary_small_te
Saved: /content/drive/MyDrive/ms_preprocessed/_slice_analysis_train20_ep500_lr3e-4_wd1e-6_dicebce/slice_level_summary_small_test
```

Preprocess ALL missing patients

```
RAW_ROOT = "/content/drive/MyDrive/50 patients"
PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"

ALL_RAW = [
    d for d in os.listdir(RAW_ROOT)
    if os.path.isdir(os.path.join(RAW_ROOT, d))
]

ALL_PREP = [
    d for d in os.listdir(PREP_ROOT)
    if os.path.isdir(os.path.join(PREP_ROOT, d)) and not d.startswith("_")
]

MISSING = sorted([pid for pid in ALL_RAW if pid not in ALL_PREP])

print("Patients to preprocess:", len(MISSING))
print(MISSING)
```

```
Patients to preprocess: 35
['00408990', '01001835', '01019493 (1)', '01138125', '01153181', '01207768', '01227551', '01228144', '01228578', '01267851', '01
```

Clean the list

```
RAW_ROOT = "/content/drive/MyDrive/50 patients"
PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"

ALL_RAW = sorted([
    d for d in os.listdir(RAW_ROOT)
    if os.path.isdir(os.path.join(RAW_ROOT, d))
    and not d.startswith("_")
    and d not in ["processed", "design1_models"]
])

ALL_PREP = sorted([
    d for d in os.listdir(PREP_ROOT)
```

```
    if os.path.isdir(os.path.join(PREP_ROOT, d))
    and not d.startswith("_")
])

MISSING = sorted([
    pid for pid in ALL_RAW
    if pid not in ALL_PREP
    and " (" not in pid   # removes copies like 01019493 (1)
])

print("Real patients to preprocess:", len(MISSING))
print(MISSING)
```

```
Real patients to preprocess: 32
['00408990', '01001835', '01138125', '01153181', '01207768', '01227551', '01228144', '01228578', '01267851', '01270947', '012966
```

Preprocess these 32 patients

```
!pip -q install SimpleITK

import os, re, glob
import numpy as np
import SimpleITK as sitk

# ---------------- PATHS ----------------
ROOTS = [
    "/content/drive/MyDrive/14 MS patients",
    "/content/drive/MyDrive/50 patients",
]

OUT_ROOT = "/content/drive/MyDrive/ms_preprocessed"
os.makedirs(OUT_ROOT, exist_ok=True)

# ---------------- NEW LIST: MISSING PATIENTS ----------------
MISSING = [
    '00408990', '01001835', '01138125', '01153181', '01207768',
    '01227551', '01228144', '01228578', '01267851', '01270947',
    '01296641', '01339318', '01344041', '01350625', '01355333',
    '01356881', '01365320', '01383096', '01390427', '01391956',
    '01397436', '01401875', '01433180', '01433317', '01433441',
    '01434978', '01454108', '01458991', '01474239', '01477668',
    '01480045', '01480874'
]

TARGET_SPACING = (0.5, 0.5, 0.5)   # (x,y,z)
TARGET_SHAPE   = (384, 512, 512)   # (x,y,z) PAD ONLY

IGNORE_IDS = {"00408990"}   # keep if you still want to skip it

# ---------------- HELPERS ----------------
def parse_date(name: str):
    m = re.search(r"(19|20)\d{6}", name)
    return int(m.group(0)) if m else None

def find_patient_dir(pid: str):
    for root in ROOTS:
        pdir = os.path.join(root, pid)
        if os.path.isdir(pdir):
            return pdir
    return None

def pick_patient_files(pid: str):
    if pid in IGNORE_IDS:
        raise FileNotFoundError(f"{pid} is ignored")

    pdir = find_patient_dir(pid)
    if pdir is None:
        raise FileNotFoundError(f"Patient folder not found in ROOTS: {pid}")

    nii = sorted(glob.glob(os.path.join(pdir, "*.nii.gz")))
    if not nii:
        raise FileNotFoundError(f"No .nii.gz files found in {pdir}")

    # label
    label_candidates = [
```

```
            f for f in nii
            if os.path.basename(f).lower().startswith("segmentation")
            and "label" in os.path.basename(f).lower()
        ]
        if not label_candidates:
            raise FileNotFoundError(f"No segmentation label file found for {pid}")
        label = sorted(label_candidates)[0]

        # registered follow-up
        reg_candidates = [
            f for f in nii
            if re.search(r"_to_FLAIR_?1\.nii\.gz$", os.path.basename(f), flags=re.IGNORECASE)
        ]
        if not reg_candidates:
            raise FileNotFoundError(f"No registered follow-up '*_to_FLAIR1.nii.gz' or '*_to_FLAIR_1.nii.gz' for {pid}")
        follow_reg = reg_candidates[0]

        # baseline = earliest dated scan excluding label + follow_reg
        dated = []
        for f in nii:
            if f == label or f == follow_reg:
                continue
            d = parse_date(os.path.basename(f))
            if d is not None:
                dated.append((d, f))

        if not dated:
            raise FileNotFoundError(f"No dated scan found for baseline for {pid}")

        dated.sort(key=lambda x: x[0])
        baseline = dated[0][1]

        return pdir, baseline, follow_reg, label

    def resample_to_spacing(img, out_spacing, interpolator, default_value=0.0):
        in_spacing = img.GetSpacing()
        in_size    = img.GetSize()

        out_size = [
            int(np.round(in_size[i] * (in_spacing[i] / out_spacing[i])))
            for i in range(3)
        ]

        ref = sitk.Image(out_size, img.GetPixelID())
        ref.SetSpacing(out_spacing)
        ref.SetDirection(img.GetDirection())
        ref.SetOrigin(img.GetOrigin())

        return sitk.Resample(img, ref, sitk.Transform(), interpolator, default_value, img.GetPixelID())

    def center_pad_only(img, target_size_xyz, pad_value=0.0):
        size = np.array(list(img.GetSize()), dtype=int)
        target = np.array(list(target_size_xyz), dtype=int)

        if np.any(size > target):
            raise ValueError(
                f"Resampled size {tuple(size)} exceeds TARGET_SHAPE {tuple(target)} "
                f"(cropping would be needed, not allowed)."
            )

        pad_lower = (target - size) // 2
        pad_upper = (target - size) - pad_lower

        return sitk.ConstantPad(
            img,
            [int(p) for p in pad_lower.tolist()],
            [int(p) for p in pad_upper.tolist()],
            float(pad_value)
        )

    def simple_foreground_mask(img):
        arr = sitk.GetArrayFromImage(img).astype(np.float32)
        nz = arr[arr > 0]
        if nz.size == 0:
            m = arr != 0
        else:
            thr = np.percentile(nz, 5)
```

```
        m = arr > thr
    out = sitk.GetImageFromArray(m.astype(np.uint8))
    out.CopyInformation(img)
    return out

def zscore_in_mask(vol, mask):
    v = sitk.GetArrayFromImage(vol).astype(np.float32)
    m = sitk.GetArrayFromImage(mask) > 0
    if m.sum() == 0:
        m = v != 0
    vals = v[m]
    mu = float(vals.mean())
    sd = float(vals.std() + 1e-6)
    v = (v - mu) / sd
    out = sitk.GetImageFromArray(v)
    out.CopyInformation(vol)
    return out

# ---------------- RUN ----------------
print("TARGET_SPACING:", TARGET_SPACING)
print("TARGET_SHAPE  :", TARGET_SHAPE, "(PAD ONLY)")

done = []
failed = []

for pid in MISSING:
    print("\n" + "="*90)
    print("Preprocessing patient:", pid)

    try:
        pdir, baseline_p, followreg_p, label_p = pick_patient_files(pid)
        print("Folder  :", pdir)
        print("Baseline:", os.path.basename(baseline_p))
        print("FollowR :", os.path.basename(followreg_p))
        print("Label   :", os.path.basename(label_p))

        b  = sitk.ReadImage(baseline_p)
        fr = sitk.ReadImage(followreg_p)
        gt = sitk.ReadImage(label_p)

        # 1) resample
        b_05  = resample_to_spacing(b,  TARGET_SPACING, sitk.sitkLinear, 0.0)
        fr_05 = resample_to_spacing(fr, TARGET_SPACING, sitk.sitkLinear, 0.0)
        gt_05 = resample_to_spacing(gt, TARGET_SPACING, sitk.sitkNearestNeighbor, 0)

        # 2) pad only
        b_fix  = center_pad_only(b_05,  TARGET_SHAPE, pad_value=0.0)
        fr_fix = center_pad_only(fr_05, TARGET_SHAPE, pad_value=0.0)
        gt_fix = center_pad_only(gt_05, TARGET_SHAPE, pad_value=0)

        # 3) force GT binary
        gt_arr = sitk.GetArrayFromImage(gt_fix).astype(np.float32)
        gt_arr = (gt_arr > 0.5).astype(np.uint8)
        gt_fix_bin = sitk.GetImageFromArray(gt_arr)
        gt_fix_bin.CopyInformation(gt_fix)

        # 4) z-score with follow-up mask
        mask = simple_foreground_mask(fr_fix)
        b_norm  = zscore_in_mask(b_fix,  mask)
        fr_norm = zscore_in_mask(fr_fix, mask)

        # 5) save
        out_dir = os.path.join(OUT_ROOT, pid)
        os.makedirs(out_dir, exist_ok=True)

        sitk.WriteImage(b_norm,     os.path.join(out_dir, "baseline_0p5_norm_padded.nii.gz"))
        sitk.WriteImage(fr_norm,    os.path.join(out_dir, "followup_registered_0p5_norm_padded.nii.gz"))
        sitk.WriteImage(gt_fix_bin, os.path.join(out_dir, "label_0p5_padded.nii.gz"))

        print("Postprocessed GT voxels:", int(gt_arr.sum()))
        print("Saved to:", out_dir)
        print("Done ✅")

        done.append(pid)

    except Exception as e:
        print(f"❌ Failed for {pid}: {e}")
```

```
        failed.append((pid, str(e)))

print("\nFinished.")
print("Succeeded:", len(done))
print(done)
print("Failed:", len(failed))
print(failed)
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.6/52.6 MB 47.6 MB/s eta 0:00:00
TARGET_SPACING: (0.5, 0.5, 0.5)
TARGET_SHAPE  : (384, 512, 512) (PAD ONLY)

================================================================================
Preprocessing patient: 00408990
❌ Failed for 00408990: 00408990 is ignored

================================================================================
Preprocessing patient: 01001835
Folder  : /content/drive/MyDrive/50 patients/01001835
Baseline: 01001835_20160923.nii.gz
FollowR : 01001835_20171026_to_FLAIR1.nii.gz
Label   : Segmentation-label.nii.gz
Postprocessed GT voxels: 31662
Saved to: /content/drive/MyDrive/ms_preprocessed/01001835
Done ✅

================================================================================
Preprocessing patient: 01138125
Folder  : /content/drive/MyDrive/50 patients/01138125
Baseline: 01138125_20160615.nii.gz
FollowR : 01138125_20170103_to_FLAIR1.nii.gz
Label   : Segmentation-Segment_1-label.nii.gz
Postprocessed GT voxels: 4160
Saved to: /content/drive/MyDrive/ms_preprocessed/01138125
Done ✅

================================================================================
Preprocessing patient: 01153181
Folder  : /content/drive/MyDrive/50 patients/01153181
Baseline: 01153181_20230725.nii.gz
FollowR : 01153181_20240802_to_FLAIR1.nii.gz
Label   : Segmentation-Segment_1-label.nii.gz
❌ Failed for 01153181: Resampled size (np.int64(384), np.int64(523), np.int64(500)) exceeds TARGET_SHAPE (np.int64(384), np.i

================================================================================
Preprocessing patient: 01207768
Folder  : /content/drive/MyDrive/50 patients/01207768
Baseline: 01207768_20160329.nii.gz
FollowR : 01207768_20170310_to_FLAIR1.nii.gz
Label   : Segmentation-Segment_1-label.nii.gz
Postprocessed GT voxels: 1664
Saved to: /content/drive/MyDrive/ms_preprocessed/01207768
Done ✅

================================================================================
Preprocessing patient: 01227551
Folder  : /content/drive/MyDrive/50 patients/01227551
Baseline: 01227551_20160812.nii.gz
FollowR : 01227551_20161215_to_FLAIR1.nii.gz
Label   : Segmentation-Segment_1-label.nii.gz
Postprocessed GT voxels: 2530
Saved to: /content/drive/MyDrive/ms_preprocessed/01227551
Done ✅

================================================================================
```

Recompute final dataset

```
ALL_PATIENTS = sorted([
    d for d in os.listdir(PREP_ROOT)
    if os.path.isdir(os.path.join(PREP_ROOT, d)) and not d.startswith("_")
])

print("Total patients:", len(ALL_PATIENTS))
```

```
Total patients: 56
```

```
USED_IDS = set(TRAIN_IDS_BASE + TEST_IDS_BASE)

REMAINING_IDS = sorted([pid for pid in ALL_PATIENTS if pid not in USED_IDS])
```

```
print("Final unseen patients:", len(REMAINING_IDS))
print(REMAINING_IDS)
```

```
Final unseen patients: 29
['01137872', '01138125', '01207768', '01227551', '01228144', '01228578', '01267851', '01270947', '01296641', '01339318', '013440
```

Run inference on all 29 patients

```
PIDS_FINAL = [
'01137872', '01138125', '01207768', '01227551', '01228144',
'01228578', '01267851', '01270947', '01296641', '01339318',
'01344041', '01350625', '01356881', '01365320', '01383096',
'01390427', '01391956', '01397436', '01401875', '01433180',
'01433317', '01433441', '01434978', '01454108', '01458991',
'01474239', '01477668', '01480045', '01480874'
]

all_patient_stats = []
all_slice_tables = []
failed_patients = []

for pid in PIDS_FINAL:
    try:
        stats, df = analyze_patient(pid, save_figures=True, max_fig_slices=None)
        all_patient_stats.append(stats)
        all_slice_tables.append(df)
    except Exception as e:
        print(f"❌ Failed on {pid}: {e}")
        failed_patients.append({"patient_id": pid, "error": str(e)})
```

```
=== Patient 01137872 ===
Dice: 0.326
GT lesion slices: [136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155] ...
Pred lesion slices: [137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 237, 238, 239, 240, 241] ...

=== Patient 01138125 ===
Dice: 0.3182
GT lesion slices: [140, 141, 142, 143, 144, 145, 146, 147, 148, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168] ...
Pred lesion slices: [134, 135, 137, 138, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 228, 229, 230, 231, 232, 233] ...

=== Patient 01207768 ===
Dice: 0.0095
GT lesion slices: [214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226]
Pred lesion slices: [198, 204, 205, 206, 207, 208, 209, 210, 216, 217, 219, 220, 236, 237]

=== Patient 01227551 ===
Dice: 0.0
GT lesion slices: [139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157]
Pred lesion slices: []

=== Patient 01228144 ===
Dice: 0.2811
GT lesion slices: [224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 242, 243, 244, 245, 246, 247, 248, 249, 250]
Pred lesion slices: [160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 223, 224, 225, 226, 227, 228, 229, 230, 231] ...

=== Patient 01228578 ===
Dice: 0.0402
GT lesion slices: [112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 131, 132, 133, 134, 135, 136, 137, 138] ...
Pred lesion slices: [222, 225, 257, 258, 259, 260, 261, 262]

=== Patient 01267851 ===
Dice: 0.6504
GT lesion slices: [167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186] ...
Pred lesion slices: [108, 109, 126, 128, 129, 135, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182] ...

=== Patient 01270947 ===
Dice: 0.5436
GT lesion slices: [127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150] ...
Pred lesion slices: [134, 135, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157] ...

=== Patient 01296641 ===
Dice: 0.2807
GT lesion slices: [165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 208, 209, 210, 211, 212, 213, 214] ...
Pred lesion slices: [211, 212, 213, 214, 215, 216, 217, 218, 219, 242, 243]

=== Patient 01339318 ===
Dice: 0.3738
```

```
GT lesion slices: [111, 112, 113, 114, 115, 116, 117, 118, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153] ...
Pred lesion slices: [198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 231, 232, 233, 234, 235] ...

=== Patient 01344041 ===
Dice: 0.5393
GT lesion slices: [153, 154, 155, 156, 157, 158, 159, 160, 189, 190, 191, 192, 193, 194, 195, 225, 226, 227, 228, 229] ...
Pred lesion slices: [232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 259, 260, 261] ...

=== Patient 01350625 ===
```

```
import os
import ast
import pandas as pd

OUT_DIR = "/content/drive/MyDrive/ms_preprocessed/_slice_analysis_train20_ep500_lr3e-4_wd1e-6_dicebce"

PIDS_FINAL = [
'01137872', '01138125', '01207768', '01227551', '01228144',
'01228578', '01267851', '01270947', '01296641', '01339318',
'01344041', '01350625', '01356881', '01365320', '01383096',
'01390427', '01391956', '01397436', '01401875', '01433180',
'01433317', '01433441', '01434978', '01454108', '01458991',
'01474239', '01477668', '01480045', '01480874'
]

# Paste the Dice values from your printed output
dice_map = {
    "01137872": 0.3260,
    "01138125": 0.3182,
    "01207768": 0.0095,
    "01227551": 0.0000,
    "01228144": 0.2811,
    "01228578": 0.0402,
    "01267851": 0.6504,
    "01270947": 0.5436,
    "01296641": 0.2807,
    "01339318": 0.3738,
    "01344041": 0.5393,
    "01350625": 0.7001,
    "01356881": 0.4138,
    "01365320": 0.0000,
    "01383096": 0.4040,
    "01390427": 0.4367,
    "01391956": 0.0000,
    "01397436": 0.2102,
    "01401875": 0.2086,
    "01433180": 0.4412,
    "01433317": 0.2743,
    "01433441": 0.1853,
    "01434978": 0.5675,
    "01454108": 0.2469,
    "01458991": 0.5266,
    "01474239": 0.0325,
    "01477668": 0.4590,
    "01480045": 0.3167,
    "01480874": 0.1344,
}

rows = []

for pid in PIDS_FINAL:
    csv_path = os.path.join(OUT_DIR, f"{pid}_slice_summary.csv")
    df = pd.read_csv(csv_path)

    gt_slices = df.loc[df["gt_has_lesion"] == 1, "slice_index"].tolist()
    pred_slices = df.loc[df["pred_has_lesion"] == 1, "slice_index"].tolist()

    rows.append({
        "patient_id": pid,
        "dice": dice_map[pid],
        "n_gt_slices": len(gt_slices),
        "n_pred_slices": len(pred_slices),
        "gt_slice_list": gt_slices,
        "pred_slice_list": pred_slices,
        "csv_path": csv_path,
    })
```

```
patient_stats_df = pd.DataFrame(rows)
patient_stats_df.to_csv(os.path.join(OUT_DIR, "patient_level_summary_final.csv"), index=False)

print("Saved:", os.path.join(OUT_DIR, "patient_level_summary_final.csv"))
print(patient_stats_df.head())
```

```
Saved: /content/drive/MyDrive/ms_preprocessed/_slice_analysis_train20_ep500_lr3e-4_wd1e-6_dicebce/patient_level_summary_final.cs
  patient_id    dice  n_gt_slices  n_pred_slices  \
0   01137872  0.3260           36             27
1   01138125  0.3182           76             25
2   01207768  0.0095           13             14
3   01227551  0.0000           19              0
4   01228144  0.2811           20             21

                                    gt_slice_list  \
0  [136, 137, 138, 139, 140, 141, 142, 143, 144, ...
1  [140, 141, 142, 143, 144, 145, 146, 147, 148, ...
2  [214, 215, 216, 217, 218, 219, 220, 221, 222, ...
3  [139, 140, 141, 142, 143, 144, 145, 146, 147, ...
4  [224, 225, 226, 227, 228, 229, 230, 231, 232, ...

                                   pred_slice_list  \
0  [137, 138, 139, 140, 141, 142, 143, 144, 145, ...
1  [134, 135, 137, 138, 160, 161, 162, 163, 164, ...
2  [198, 204, 205, 206, 207, 208, 209, 210, 216, ...
3                                                 []
4  [160, 161, 162, 163, 164, 165, 166, 167, 168, ...

                                        csv_path
0  /content/drive/MyDrive/ms_preprocessed/_slice_...
1  /content/drive/MyDrive/ms_preprocessed/_slice_...
2  /content/drive/MyDrive/ms_preprocessed/_slice_...
3  /content/drive/MyDrive/ms_preprocessed/_slice_...
4  /content/drive/MyDrive/ms_preprocessed/_slice_...
```

Load and View final results

```
import pandas as pd
import os

OUT_DIR = "/content/drive/MyDrive/ms_preprocessed/_slice_analysis_train20_ep500_lr3e-4_wd1e-6_dicebce"

df = pd.read_csv(os.path.join(OUT_DIR, "patient_level_summary_final.csv"))

df
```

	patient_id	dice	n_gt_slices	n_pred_slices	gt_slice_list	pred_slice_list	csv_path
0	1137872	0.3260	36	27	[136, 137, 138, 139, 140, 141, 142, 143, 144, ...	[137, 138, 139, 140, 141, 142, 143, 144, 145, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
1	1138125	0.3182	76	25	[140, 141, 142, 143, 144, 145, 146, 147, 148, ...	[134, 135, 137, 138, 160, 161, 162, 163, 164, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
2	1207768	0.0095	13	14	[214, 215, 216, 217, 218, 219, 220, 221, 222, ...	[198, 204, 205, 206, 207, 208, 209, 210, 216, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
3	1227551	0.0000	19	0	[139, 140, 141, 142, 143, 144, 145, 146, 147, ...	[]	/content/drive/MyDrive/ms_preprocessed/_slice_...
4	1228144	0.2811	20	21	[224, 225, 226, 227, 228, 229, 230, 231, 232, ...	[160, 161, 162, 163, 164, 165, 166, 167, 168, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
5	1228578	0.0402	58	8	[112, 113, 114, 115, 116, 117, 118, 119, 120, ...	[222, 225, 257, 258, 259, 260, 261, 262]	/content/drive/MyDrive/ms_preprocessed/_slice_...
6	1267851	0.6504	43	50	[167, 168, 169, 170, 171, 172, 173, 174, 175, ...	[108, 109, 126, 128, 129, 135, 169, 170, 171, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
7	1270947	0.5436	109	97	[127, 128, 129, 130, 131, 132, 133, 134, 135, ...	[134, 135, 140, 141, 142, 143, 144, 145, 146, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
8	1296641	0.2807	27	11	[165, 166, 167, 168, 169, 170, 171, 172, 173, ...	[211, 212, 213, 214, 215, 216, 217, 218, 219, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
9	1339318	0.3738	69	30	[111, 112, 113, 114, 115, 116, 117, 118, 142, ...	[198, 199, 200, 201, 202, 203, 204, 205, 206, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
10	1344041	0.5393	51	30	[153, 154, 155, 156, 157, 158, 159, 160, 189, ...	[232, 233, 234, 235, 236, 237, 238, 239, 240, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
11	1350625	0.7001	44	35	[111, 112, 113, 114, 115, 116, 117, 118, 119, ...	[77, 78, 79, 81, 82, 111, 112, 113, 114, 115, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
12	1380796	0.4040	39	44	[125, 126, 127, ...	[230, 231, 232, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
13	1383001	0.3807	39	44	[147, 148, 149, 150, 151, 152, 153, 154, 228, ...	[166, 167, 168, 169, 170, 172, 173, 177, 178, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
14	1390427	0.4367	67	22	[135, 136, 137, 138, 139, 140, 141, 142, 143, ...	[97, 98, 213, 225, 226, 227, 228, ...	/content/drive/MyDrive/ms_preprocessed/_slice_...
15	1391956	0.0000	9	2	[146, 147, 148, 149, 150, 151]	[150, 152]	/content/drive/MyDrive/ms_preprocessed/_slice_...

Show Summary Statistics

```
print("Number of patients:", len(df))
print("Average Dice:", round(df["dice"].mean(), 4))
print("Median Dice:", round(df["dice"].median(), 4))
print("Best Dice:", round(df["dice"].max(), 4))
print("Worst Dice:", round(df["dice"].min(), 4))
```

```
Number of patients: 29
Average Dice: 0.3076
Median Dice: 0.3167
Best Dice: 0.7001
Worst Dice: 0.0
```

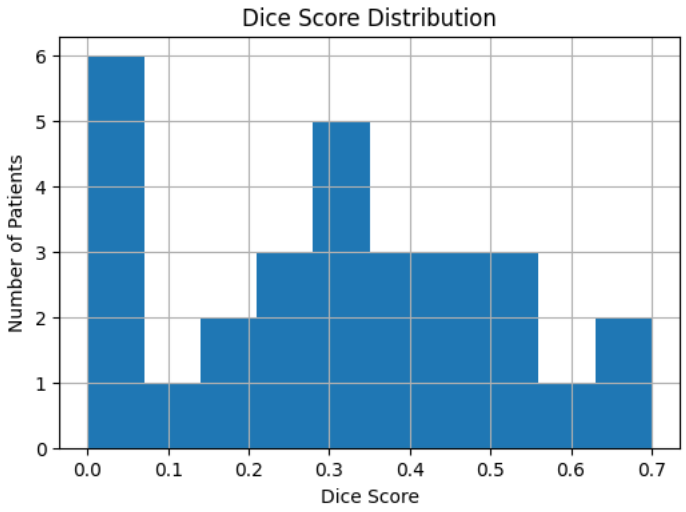
Sort Patients by Performance

```
df_sorted = df.sort_values("dice", ascending=False)
df_sorted[["patient_id", "dice"]]
```

	patient_id	dice
11	1350625	0.7001
6	1267851	0.6504
22	1434978	0.5675
7	1270947	0.5436
10	1344041	0.5393
24	1458991	0.5266
26	1477668	0.4590
19	1433180	0.4412
15	1390427	0.4367
12	1356881	0.4138
14	1383096	0.4040
9	1339318	0.3738
0	1137872	0.3260
1	1138125	0.3182
27	1480045	0.3167
4	1228144	0.2811
8	1296641	0.2807
20	1433317	0.2743
23	1454108	0.2469
17	1397436	0.2102
18	1401875	0.2086
21	1433441	0.1853
28	1480874	0.1344
5	1228578	0.0402
25	1474239	0.0325
2	1207768	0.0095
3	1227551	0.0000
13	1365320	0.0000
16	1391956	0.0000

VISUALIZATION PLOTS (FOR REPORT)

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))
plt.hist(df["dice"], bins=10)
plt.xlabel("Dice Score")
plt.ylabel("Number of Patients")
plt.title("Dice Score Distribution")
plt.grid()
plt.show()
```



VISUALIZE MRI + LESIONS (IMPORTANT)

```
import os
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def normalize_for_display(img2d):
    img2d = img2d.astype(np.float32)
    mn, mx = img2d.min(), img2d.max()
    return (img2d - mn) / (mx - mn + 1e-8)

def get_lesion_slices(gt, pred, mode="union"):
    gt_slices = set(np.where(gt.sum(axis=(1, 2)) > 0)[0].tolist())
    pred_slices = set(np.where(pred.sum(axis=(1, 2)) > 0)[0].tolist())

    if mode == "gt":
        slices = sorted(gt_slices)
    elif mode == "pred":
        slices = sorted(pred_slices)
    elif mode == "intersection":
        slices = sorted(gt_slices & pred_slices)
    else:  # union
        slices = sorted(gt_slices | pred_slices)

    return slices

def make_overlay(mri2d, gt2d, pred2d):
    mri = normalize_for_display(mri2d)
    rgb = np.stack([mri, mri, mri], axis=-1)

    gt_mask = gt2d.astype(bool)
    pred_mask = pred2d.astype(bool)

    # GT = green
    rgb[gt_mask, 1] = 1.0

    # Prediction = red
    rgb[pred_mask, 0] = 1.0

    # Overlap = yellow
    overlap = gt_mask & pred_mask
    rgb[overlap, 0] = 1.0
    rgb[overlap, 1] = 1.0
    rgb[overlap, 2] = 0.0

    return rgb

def visualize_mri_gt_pred_slices(
    pid,
    b=None,
    f=None,
    gt=None,
```

```
    pred=None,
    use_followup=True,
    mode="union",          # "gt", "pred", "intersection", "union"
    max_slices=None,
    start_from=0,
    save_dir=None,
    show=True
):
    """
    Assumes either:
    1) you pass b, f, gt, pred directly
    OR
    2) you already have load_preprocessed(pid) and get_or_make_prediction(pid)
    """

    if any(x is None for x in [b, f, gt, pred]):
        b, f, gt_loaded = load_preprocessed(pid)
        _, _, _, pred_loaded, _ = get_or_make_prediction(pid, force_recompute=False)
        gt = gt_loaded
        pred = pred_loaded

    img_vol = f if use_followup else b
    slices = get_lesion_slices(gt, pred, mode=mode)

    if start_from > 0:
        slices = slices[start_from:]

    if max_slices is not None:
        slices = slices[:max_slices]

    print(f"Patient: {pid}")
    print(f"Showing {len(slices)} slices | mode={mode}")

    if save_dir is not None:
        os.makedirs(save_dir, exist_ok=True)

    for z in slices:
        mri2d = img_vol[z]
        gt2d = gt[z]
        pred2d = pred[z]
        overlay = make_overlay(mri2d, gt2d, pred2d)

        gt_area = int(gt2d.sum())
        pred_area = int(pred2d.sum())
        overlap_area = int((gt2d.astype(bool) & pred2d.astype(bool)).sum())

        fig, axes = plt.subplots(1, 4, figsize=(18, 4))

        axes[0].imshow(normalize_for_display(mri2d), cmap="gray")
        axes[0].set_title(f"MRI\nslice {z}")
        axes[0].axis("off")

        axes[1].imshow(gt2d, cmap="gray")
        axes[1].set_title(f"GT\narea={gt_area}")
        axes[1].axis("off")

        axes[2].imshow(pred2d, cmap="gray")
        axes[2].set_title(f"Prediction\narea={pred_area}")
        axes[2].axis("off")

        axes[3].imshow(overlay)
        axes[3].set_title(f"Overlay\noverlap={overlap_area}")
        axes[3].axis("off")

        plt.tight_layout()

        if save_dir is not None:
            plt.savefig(os.path.join(save_dir, f"{pid}_slice_{z:03d}.png"),
                        dpi=180, bbox_inches="tight")

        if show:
            plt.show()
        else:
            plt.close(fig)
```

```
import os
import numpy as np
import torch

CACHE_DIR = "/content/drive/MyDrive/ms_preprocessed/_pred_cache"
os.makedirs(CACHE_DIR, exist_ok=True)

PATCH_SIZE = (64, 64, 64)
STRIDE = (32, 32, 32)
THR = 0.5

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def get_or_make_prediction(pid, thr=THR, force_recompute=False):
    pid = str(pid).zfill(8)

    pred_path = os.path.join(CACHE_DIR, f"{pid}_pred.npy")
    prob_path = os.path.join(CACHE_DIR, f"{pid}_prob.npy")

    b, f, gt = load_preprocessed(pid)

    if (not force_recompute) and os.path.exists(pred_path):
        pred = np.load(pred_path)
        prob = np.load(prob_path) if os.path.exists(prob_path) else None
        return b, f, gt, pred, prob

    print(f"Running inference for {pid} ...")
    prob = sliding_window_predict(
        model, b, f,
        patch_size=PATCH_SIZE,
        stride=STRIDE
    )
    pred = (prob >= thr).astype(np.uint8)

    np.save(pred_path, pred)
    np.save(prob_path, prob)

    return b, f, gt, pred, prob
```

Last PostProcessing Visualization

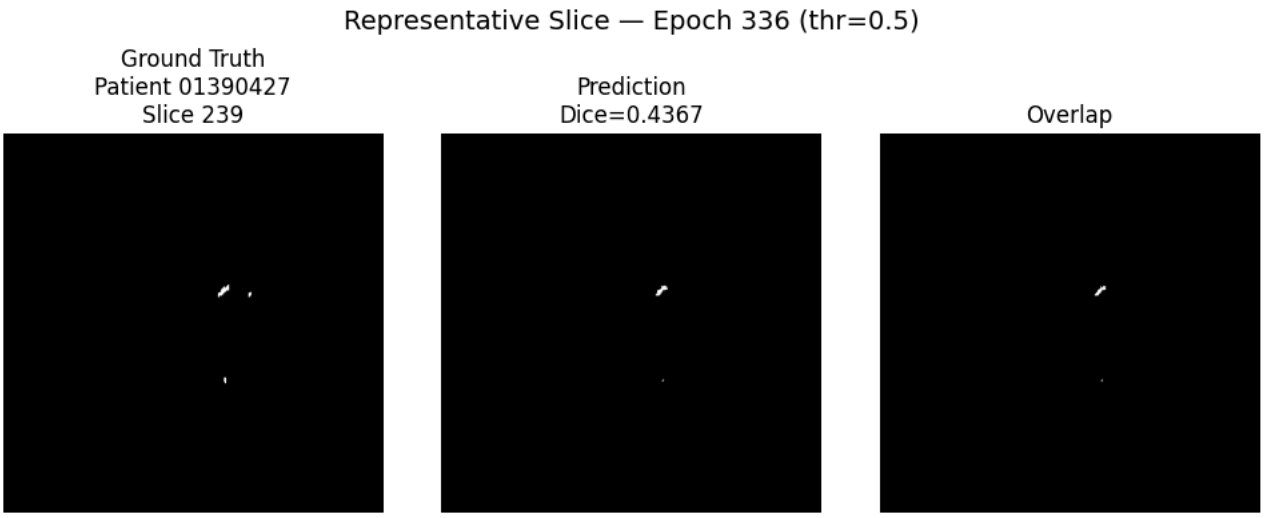
```
import os
import numpy as np
import matplotlib.pyplot as plt

def visualize_and_save_all_slices_for_patient(
    pid,
    mode="gt",   # "gt", "pred", "union", "intersection"
    use_followup=True,
    thr=0.5,
    base_save_dir="/content/drive/MyDrive/ms_preprocessed/_postprocessing_results",
    show=True
):
    pid = str(pid).zfill(8)

    # folder for this patient
    patient_dir = os.path.join(base_save_dir, pid)
    os.makedirs(patient_dir, exist_ok=True)

    print(f"Saving results for patient {pid} -> {patient_dir}")

    # load data + prediction
    b, f, gt = load_preprocessed(pid)
    _, _, _, _, prob = get_or_make_prediction(pid, force_recompute=False)
    pred = (prob >= thr).astype(np.uint8)

    img_vol = f if use_followup else b

    # choose slice list
    gt_slices = set(np.where(gt.sum(axis=(1, 2)) > 0)[0].tolist())
    pred_slices = set(np.where(pred.sum(axis=(1, 2)) > 0)[0].tolist())

    if mode == "gt":
        slices = sorted(gt_slices)
    elif mode == "pred":
```

```
        slices = sorted(pred_slices)
    elif mode == "intersection":
        slices = sorted(gt_slices & pred_slices)
    else:
        slices = sorted(gt_slices | pred_slices)

    print(f"Total slices to process: {len(slices)}")
    print("Slice list:", slices)

    def normalize(img):
        img = img.astype(np.float32)
        return (img - img.min()) / (img.max() - img.min() + 1e-8)

    def make_overlay(mri, gt2d, pred2d):
        mri = normalize(mri)
        rgb = np.stack([mri, mri, mri], axis=-1)

        gt_mask = gt2d.astype(bool)
        pred_mask = pred2d.astype(bool)
        overlap = gt_mask & pred_mask

        rgb[gt_mask, 1] = 1.0      # GT green
        rgb[pred_mask, 0] = 1.0    # Pred red
        rgb[overlap, 0] = 1.0
        rgb[overlap, 1] = 1.0
        rgb[overlap, 2] = 0.0      # overlap yellow

        return rgb

    for z in slices:
        mri2d = img_vol[z]
        gt2d = gt[z]
        pred2d = pred[z]
        overlay = make_overlay(mri2d, gt2d, pred2d)

        gt_area = int(gt2d.sum())
        pred_area = int(pred2d.sum())
        overlap_area = int((gt2d.astype(bool) & pred2d.astype(bool)).sum())

        fig, axes = plt.subplots(1, 4, figsize=(15, 4))

        axes[0].imshow(normalize(mri2d), cmap="gray")
        axes[0].set_title(f"MRI\nslice {z}")
        axes[0].axis("off")

        axes[1].imshow(gt2d, cmap="gray")
        axes[1].set_title(f"GT\narea={gt_area}")
        axes[1].axis("off")

        axes[2].imshow(pred2d, cmap="gray")
        axes[2].set_title(f"Pred\narea={pred_area}")
        axes[2].axis("off")

        axes[3].imshow(overlay)
        axes[3].set_title(f"Overlay\noverlap={overlap_area}")
        axes[3].axis("off")

        plt.tight_layout()

        save_path = os.path.join(patient_dir, f"{pid}_slice_{z:03d}.png")
        plt.savefig(save_path, dpi=150, bbox_inches="tight")

        if show:
            plt.show()

        plt.close(fig)

    print("Done.")
    print("Saved folder:", patient_dir)
```

```
visualize_and_save_all_slices_for_patient(
    pid="01390427",
    mode="gt",
    thr=0.5,
    show=True
)
```

```
plot_one_patient_one_epoch("01390427", epoch=336, thr=0.5)
```

```
Loaded: /content/drive/MyDrive/ms_preprocessed/_checkpoints/train20_ep500_lr3e-4_wd1e-6_dicebce/epoch_336.pt
```




```
Saved: /content/drive/MyDrive/ms_preprocessed/_visual_results_train20_ep500_lr3e-4_wd1e-6_dicebce/01390427_epoch_336_repr.png
Representative slice = 239
Patient Dice = 0.4367
```

Run 28 patients

```
import os
import numpy as np
import matplotlib.pyplot as plt

def visualize_and_save_all_slices_for_patient(
    pid,
    mode="gt",
    use_followup=True,
    thr=0.5,
    base_save_dir="/content/drive/MyDrive/ms_preprocessed/_postprocessing_results",
    show=True
):
    pid = str(pid).zfill(8)

    patient_dir = os.path.join(base_save_dir, pid)
    os.makedirs(patient_dir, exist_ok=True)

    print(f"\nSaving results for patient {pid} -> {patient_dir}")

    b, f, gt = load_preprocessed(pid)
    _, _, _, _, prob = get_or_make_prediction(pid, force_recompute=False)
    pred = (prob >= thr).astype(np.uint8)

    img_vol = f if use_followup else b

    gt_slices = set(np.where(gt.sum(axis=(1, 2)) > 0)[0].tolist())
    pred_slices = set(np.where(pred.sum(axis=(1, 2)) > 0)[0].tolist())

    if mode == "gt":
        slices = sorted(gt_slices)
    elif mode == "pred":
        slices = sorted(pred_slices)
    elif mode == "intersection":
        slices = sorted(gt_slices & pred_slices)
    else:
        slices = sorted(gt_slices | pred_slices)

    print(f"Total slices to process: {len(slices)}")

    def normalize(img):
        img = img.astype(np.float32)
        return (img - img.min()) / (img.max() - img.min() + 1e-8)
```

```
    def make_overlay(mri, gt2d, pred2d):
        mri = normalize(mri)
        rgb = np.stack([mri, mri, mri], axis=-1)

        gt_mask = gt2d.astype(bool)
        pred_mask = pred2d.astype(bool)
        overlap = gt_mask & pred_mask

        rgb[gt_mask, 1] = 1.0      # GT green
        rgb[pred_mask, 0] = 1.0    # Pred red
        rgb[overlap, 0] = 1.0
        rgb[overlap, 1] = 1.0
        rgb[overlap, 2] = 0.0      # overlap yellow

        return rgb

    for z in slices:
        mri2d = img_vol[z]
        gt2d = gt[z]
        pred2d = pred[z]
        overlay = make_overlay(mri2d, gt2d, pred2d)

        gt_area = int(gt2d.sum())
        pred_area = int(pred2d.sum())
        overlap_area = int((gt2d.astype(bool) & pred2d.astype(bool)).sum())

        fig, axes = plt.subplots(1, 4, figsize=(15, 4))

        axes[0].imshow(normalize(mri2d), cmap="gray")
        axes[0].set_title(f"MRI\nslice {z}")
        axes[0].axis("off")

        axes[1].imshow(gt2d, cmap="gray")
        axes[1].set_title(f"GT\narea={gt_area}")
        axes[1].axis("off")

        axes[2].imshow(pred2d, cmap="gray")
        axes[2].set_title(f"Pred\narea={pred_area}")
        axes[2].axis("off")

        axes[3].imshow(overlay)
        axes[3].set_title(f"Overlay\noverlap={overlap_area}")
        axes[3].axis("off")

        plt.tight_layout()

        save_path = os.path.join(patient_dir, f"{pid}_slice_{z:03d}.png")
        plt.savefig(save_path, dpi=150, bbox_inches="tight")

        if show:
            plt.show()

        plt.close(fig)

    print("Done.")
    print("Saved folder:", patient_dir)
```

Define all patients

```
all_patients = [
    "01137872", "01138125", "01207768", "01227551", "01228144",
    "01228578", "01267851", "01270947", "01296641", "01339318",
    "01344041", "01350625", "01356881", "01365320", "01383096",
    "01390427", "01391956", "01397436", "01401875", "01433180",
    "01433317", "01433441", "01434978", "01454108", "01458991",
    "01474239", "01477668", "01480045", "01480874"
]
```

choose the 4 patients you want to DISPLAY

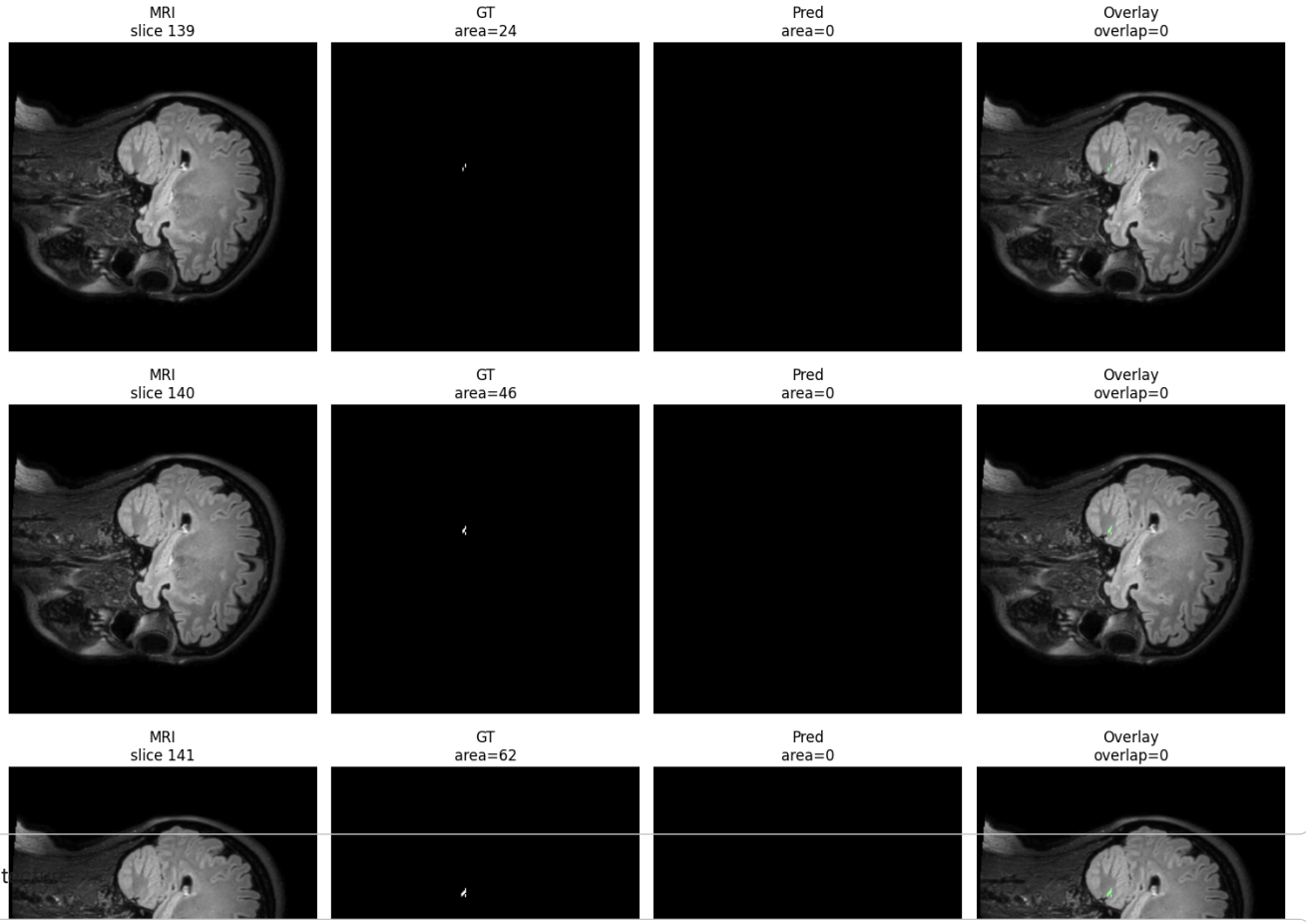
```
patients_to_view = [
    "01350625",  # best
    "01267851",  # good
    "01390427",  # semi-good
```

```
    "01227551"   # worst
]
```

save for ALL patients, but show only the 4 selected ones

```
for pid in all_patients:
    show_flag = pid in patients_to_view

    visualize_and_save_all_slices_for_patient(
        pid=pid,
        mode="gt",
        thr=0.5,
        show=show_flag
    )
```

```
Saving results for patient 01137872 -> /content/drive/MyDrive/ms_preprocessed/_postprocessing_results/01137872
Running inference for 01137872 ...
Total slices to process: 36
Done.
Saved folder: /content/drive/MyDrive/ms_preprocessed/_postprocessing_results/01137872

Saving results for patient 01138125 -> /content/drive/MyDrive/ms_preprocessed/_postprocessing_results/01138125
Running inference for 01138125 ...
Total slices to process: 76
Done.
Saved folder: /content/drive/MyDrive/ms_preprocessed/_postprocessing_results/01138125

Saving results for patient 01207768 -> /content/drive/MyDrive/ms_preprocessed/_postprocessing_results/01207768
Running inference for 01207768 ...
Total slices to process: 13
Done.
Saved folder: /content/drive/MyDrive/ms_preprocessed/_postprocessing_results/01207768

Saving results for patient 01227551 -> /content/drive/MyDrive/ms_preprocessed/_postprocessing_results/01227551
Total slices to process: 19
```

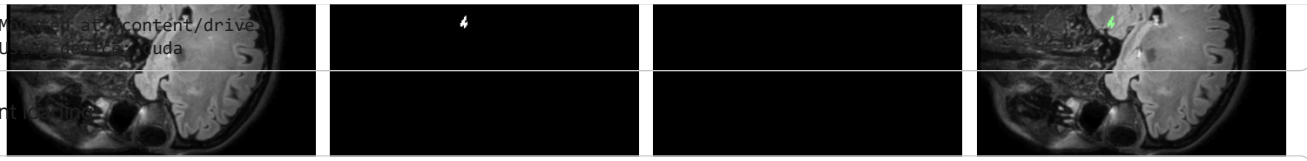


Archit

```
# ========================================
# CELL 1: SETUP
# ========================================
import os
```

```
import numpy as np
import pandas as pd
import nibabel as nib
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F

from google.colab import drive

# Mount Drive
drive.mount('/content/drive')

# Paths
PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"
CHECKPOINT_ROOT = os.path.join(PREP_ROOT, "_checkpoints")

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Mounted at /content/drive
Using device: cuda
```



Patient ID: 1

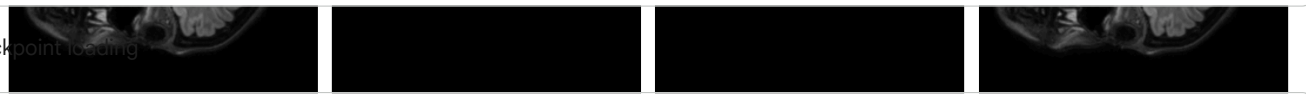
```
# ========================================
# CELL 2: LOAD PREPROCESSED PATIENT
# ========================================
B_NAME  = "baseline_0p5_norm_padded.nii.gz"
F_NAME  = "followup_registered_0p5_norm_padded.nii.gz"
Y_NAME  = "label_0p5_padded.nii.gz"

def load_preprocessed(pid):
    patient_dir = os.path.join(PREP_ROOT, pid)

    b_path = os.path.join(patient_dir, B_NAME)
    f_path = os.path.join(patient_dir, F_NAME)
    y_path = os.path.join(patient_dir, Y_NAME)

    if not os.path.exists(b_path):
        raise FileNotFoundError(f"Missing baseline file: {b_path}")
    if not os.path.exists(f_path):
        raise FileNotFoundError(f"Missing followup file: {f_path}")
    if not os.path.exists(y_path):
        raise FileNotFoundError(f"Missing label file: {y_path}")

    b = nib.load(b_path).get_fdata().astype(np.float32)
    f = nib.load(f_path).get_fdata().astype(np.float32)
    y = nib.load(y_path).get_fdata().astype(np.float32)

    return b, f, y
```

Current/original architecture

```
# ========================================
# CELL 3: ORIGINAL MODEL ARCHITECTURE
# ========================================
class ConvBlock3D(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv3d(in_ch, out_ch, kernel_size=3, padding=1),
            nn.InstanceNorm3d(out_ch),
            nn.LeakyReLU(inplace=True),

            nn.Conv3d(out_ch, out_ch, kernel_size=3, padding=1),
            nn.InstanceNorm3d(out_ch),
            nn.LeakyReLU(inplace=True),
        )

    def forward(self, x):
        return self.block(x)
```

```
class ModifiedUNet3D(nn.Module):
    def __init__(self, in_channels=2, out_channels=1, base_ch=16):
        super().__init__()

        self.enc1 = ConvBlock3D(in_channels, base_ch)
        self.pool1 = nn.MaxPool3d(2)

        self.enc2 = ConvBlock3D(base_ch, base_ch*2)
        self.pool2 = nn.MaxPool3d(2)

        self.enc3 = ConvBlock3D(base_ch*2, base_ch*4)
        self.pool3 = nn.MaxPool3d(2)

        self.bottleneck = ConvBlock3D(base_ch*4, base_ch*8)

        self.up3 = nn.ConvTranspose3d(base_ch*8, base_ch*4, kernel_size=2, stride=2)
        self.dec3 = ConvBlock3D(base_ch*8, base_ch*4)

        self.up2 = nn.ConvTranspose3d(base_ch*4, base_ch*2, kernel_size=2, stride=2)
        self.dec2 = ConvBlock3D(base_ch*4, base_ch*2)

        self.up1 = nn.ConvTranspose3d(base_ch*2, base_ch, kernel_size=2, stride=2)
        self.dec1 = ConvBlock3D(base_ch*2, base_ch)

        self.out_conv = nn.Conv3d(base_ch, out_channels, kernel_size=1)

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool1(e1))
        e3 = self.enc3(self.pool2(e2))

        b = self.bottleneck(self.pool3(e3))

        d3 = self.up3(b)
        d3 = torch.cat([d3, e3], dim=1)
        d3 = self.dec3(d3)

        d2 = self.up2(d3)
        d2 = torch.cat([d2, e2], dim=1)
        d2 = self.dec2(d2)

        d1 = self.up1(d2)
        d1 = torch.cat([d1, e1], dim=1)
        d1 = self.dec1(d1)

        return self.out_conv(d1)
```

Checkpoint loading



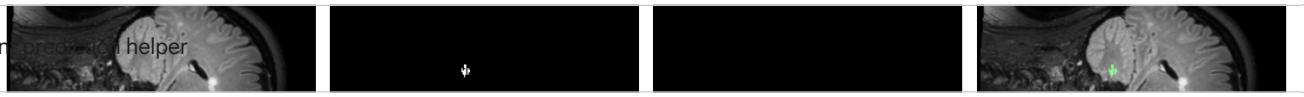
```
# ========================================
# CELL 4: LOAD CHECKPOINT
# ========================================
def load_model_from_checkpoint(checkpoint_path, model_class=ModifiedUNet3D, base_ch=16):
    model = model_class(in_channels=2, out_channels=1, base_ch=base_ch).to(device)

    ckpt = torch.load(checkpoint_path, map_location=device)
    if isinstance(ckpt, dict) and "model_state_dict" in ckpt:
        model.load_state_dict(ckpt["model_state_dict"])
    else:
        model.load_state_dict(ckpt)

    model.eval()
    return model
```

Sliding window inference


```
# ========================================
# CELL 5: SLIDING WINDOW INFERENCE
# ========================================
def sliding_window_predict(model, volume, patch_size=(64,64,64), stride=(32,32,32)):
    model.eval()
```

```
    C, H, W, D = volume.shape
    ph, pw, pd = patch_size
    sh, sw, sd = stride

    output = np.zeros((H, W, D), dtype=np.float32)
    count_map = np.zeros((H, W, D), dtype=np.float32)

    with torch.no_grad():
        for i in range(0, H - ph + 1, sh):
            for j in range(0, W - pw + 1, sw):
                for k in range(0, D - pd + 1, sd):
                    patch = volume[:, i:i+ph, j:j+pw, k:k+pd]
                    patch_tensor = torch.from_numpy(patch).unsqueeze(0).to(device)

                    pred = model(patch_tensor)
                    pred = torch.sigmoid(pred).cpu().numpy()[0, 0]

                    output[i:i+ph, j:j+pw, k:k+pd] += pred
                    count_map[i:i+ph, j:j+pw, k:k+pd] += 1

    output = output / np.maximum(count_map, 1e-8)
    return output
```

Patien prediction helper

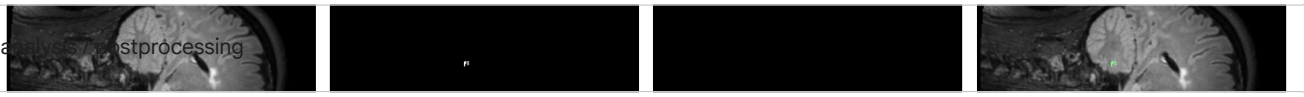


```
# ========================================
# CELL 6: PREDICT ONE PATIENT
# ========================================
def predict_patient(pid, model, threshold=0.5):
    b, f, y = load_preprocessed(pid)

    x = np.stack([b, f], axis=0)  # [C,H,W,D]
    prob = sliding_window_predict(model, x)
    pred = (prob >= threshold).astype(np.uint8)

    return b, f, y, prob, pred
```

Dice m

```
# ========================================
# CELL 7: DICE SCORE
# ========================================
def dice_score(gt, pred, eps=1e-8):
    gt = gt.astype(np.float32)
    pred = pred.astype(np.float32)

    intersection = np.sum(gt * pred)
    return (2.0 * intersection) / (np.sum(gt) + np.sum(pred) + eps)
```

Slice a stprocessing



```
# ========================================
# CELL 8: SLICE ANALYSIS
# ========================================
def get_lesion_slices(mask):
    slices = []
    for z in range(mask.shape[2]):
        if np.sum(mask[:, :, z]) > 0:
            slices.append(z)
    return slices

def summarize_patient(pid, model, threshold=0.5):
    b, f, gt, prob, pred = predict_patient(pid, model, threshold)

    gt_slices = get_lesion_slices(gt)
    pred_slices = get_lesion_slices(pred)

    summary = {
        "patient_id": pid,
        "dice": round(dice_score(gt, pred), 4),
        "n_gt_slices": len(gt_slices),
        "n_pred_slices": len(pred_slices),
```

```
        "gt_slice_list": gt_slices,
        "pred_slice_list": pred_slices,
    }
    return summary, b, f, gt, prob, pred
```

```
Saved folder: /content/drive/MyDrive/ms_preprocessed/_postprocessing_results/01228144
```

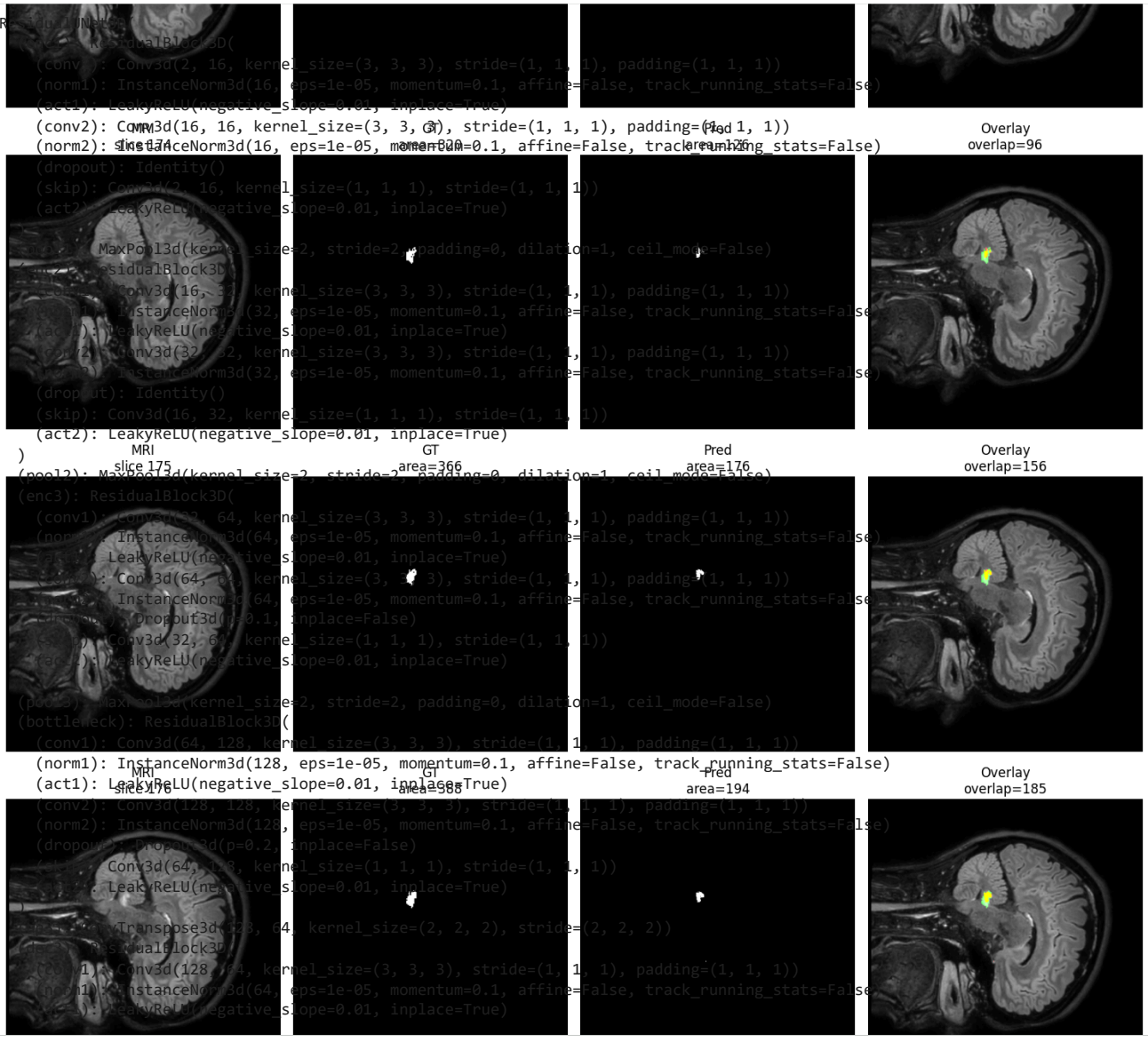
NEW ARCHITECTURE: RESIDUAL 3D U-NET

```
Saving results for patient 01228578 -> /content/drive/MyDrive/ms_preprocessed/_postprocessing_results/01228578
Running inference for 01228578 ...
```

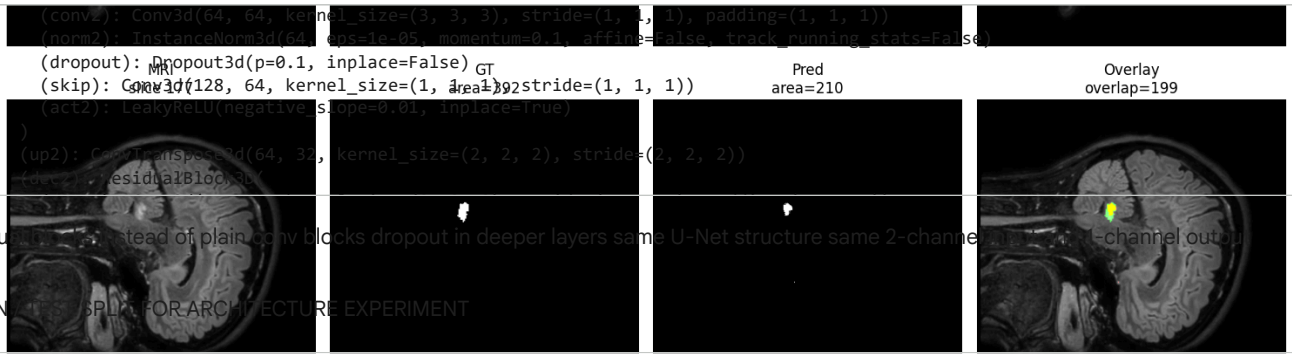
```
# ========================================
# CELL 9: NEW ARCHITECTURE - RESIDUAL 3D U-NET
# ========================================
import torch
import torch.nn as nn
import torch.nn.functional as F

class ResidualBlock3D(nn.Module):
    def __init__(self, in_ch, out_ch, dropout=0.0):
        super().__init__()

        self.conv1 = nn.Conv3d(in_ch, out_ch, kernel_size=3, padding=1)
        self.norm1 = nn.InstanceNorm3d(out_ch)
        self.act1 = nn.LeakyReLU(inplace=True)

        self.conv2 = nn.Conv3d(out_ch, out_ch, kernel_size=3, padding=1)
        self.norm2 = nn.InstanceNorm3d(out_ch)

        self.dropout = nn.Dropout3d(dropout) if dropout > 0 else nn.Identity()

        if in_ch != out_ch:
            self.skip = nn.Conv3d(in_ch, out_ch, kernel_size=1)
        else:
            self.skip = nn.Identity()

        self.act2 = nn.LeakyReLU(inplace=True)

    def forward(self, x):
        identity = self.skip(x)

        out = self.conv1(x)
        out = self.norm1(out)
        out = self.act1(out)

        out = self.conv2(out)
        out = self.norm2(out)
        out = self.dropout(out)

        out = out + identity
        out = self.act2(out)
        return out


class ResidualUNet3D(nn.Module):
    def __init__(self, in_channels=2, out_channels=1, base_ch=16, dropout=0.2):
        super().__init__()

        self.enc1 = ResidualBlock3D(in_channels, base_ch, dropout=0.0)
        self.pool1 = nn.MaxPool3d(2)

        self.enc2 = ResidualBlock3D(base_ch, base_ch * 2, dropout=0.0)
        self.pool2 = nn.MaxPool3d(2)

        self.enc3 = ResidualBlock3D(base_ch * 2, base_ch * 4, dropout=0.1)
        self.pool3 = nn.MaxPool3d(2)

        self.bottleneck = ResidualBlock3D(base_ch * 4, base_ch * 8, dropout=dropout)

        self.up3 = nn.ConvTranspose3d(base_ch * 8, base_ch * 4, kernel_size=2, stride=2)
        self.dec3 = ResidualBlock3D(base_ch * 8, base_ch * 4, dropout=0.1)

        self.up2 = nn.ConvTranspose3d(base_ch * 4, base_ch * 2, kernel_size=2, stride=2)
        self.dec2 = ResidualBlock3D(base_ch * 4, base_ch * 2, dropout=0.0)

        self.up1 = nn.ConvTranspose3d(base_ch * 2, base_ch, kernel_size=2, stride=2)
        self.dec1 = ResidualBlock3D(base_ch * 2, base_ch, dropout=0.0)

        self.out_conv = nn.Conv3d(base_ch, out_channels, kernel_size=1)
```

```
    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool1(e1))
        e3 = self.enc3(self.pool2(e2))

        b = self.bottleneck(self.pool3(e3))

        d3 = self.up3(b)
        if d3.shape[2:] != e3.shape[2:]:
            d3 = F.interpolate(d3, size=e3.shape[2:], mode="trilinear", align_corners=False)
        d3 = torch.cat([d3, e3], dim=1)
        d3 = self.dec3(d3)

        d2 = self.up2(d3)
        if d2.shape[2:] != e2.shape[2:]:
            d2 = F.interpolate(d2, size=e2.shape[2:], mode="trilinear", align_corners=False)
        d2 = torch.cat([d2, e2], dim=1)
        d2 = self.dec2(d2)

        d1 = self.up1(d2)
        if d1.shape[2:] != e1.shape[2:]:
            d1 = F.interpolate(d1, size=e1.shape[2:], mode="trilinear", align_corners=False)
        d1 = torch.cat([d1, e1], dim=1)
        d1 = self.dec1(d1)

        return self.out_conv(d1)

# Build model
model = ResidualUNet3D(in_channels=2, out_channels=1, base_ch=16, dropout=0.2).to(device)
print(model)
```

```
ResidualUNet3D(
  (enc1): ResidualBlock3D(
    (conv1): Conv3d(2, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm1): InstanceNorm3d(16, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (act1): LeakyReLU(negative_slope=0.01, inplace=True)
    (conv2): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm2): InstanceNorm3d(16, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (dropout): Identity()
    (skip): Conv3d(2, 16, kernel_size=(1, 1, 1), stride=(1, 1, 1))
    (act2): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (pool1): MaxPool3d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (enc2): ResidualBlock3D(
    (conv1): Conv3d(16, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm1): InstanceNorm3d(32, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (act1): LeakyReLU(negative_slope=0.01, inplace=True)
    (conv2): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm2): InstanceNorm3d(32, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (dropout): Identity()
    (skip): Conv3d(16, 32, kernel_size=(1, 1, 1), stride=(1, 1, 1))
    (act2): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (pool2): MaxPool3d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (enc3): ResidualBlock3D(
    (conv1): Conv3d(32, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm1): InstanceNorm3d(64, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (act1): LeakyReLU(negative_slope=0.01, inplace=True)
    (conv2): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm2): InstanceNorm3d(64, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (dropout): Dropout3d(p=0.1, inplace=False)
    (skip): Conv3d(32, 64, kernel_size=(1, 1, 1), stride=(1, 1, 1))
    (act2): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (pool3): MaxPool3d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (bottleneck): ResidualBlock3D(
    (conv1): Conv3d(64, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm1): InstanceNorm3d(128, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (act1): LeakyReLU(negative_slope=0.01, inplace=True)
    (conv2): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm2): InstanceNorm3d(128, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (dropout): Dropout3d(p=0.2, inplace=False)
    (skip): Conv3d(64, 128, kernel_size=(1, 1, 1), stride=(1, 1, 1))
    (act2): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (up3): ConvTranspose3d(128, 64, kernel_size=(2, 2, 2), stride=(2, 2, 2))
  (dec3): ResidualBlock3D(
    (conv1): Conv3d(128, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm1): InstanceNorm3d(64, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (act1): LeakyReLU(negative_slope=0.01, inplace=True)
```



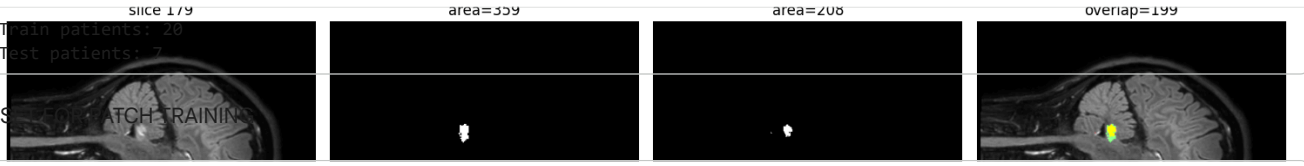
```
    (conv2): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (norm2): InstanceNorm3d(64, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
    (dropout): Dropout3d(p=0.1, inplace=False)
    (skip): Conv3d(128, 64, kernel_size=(1, 1, 1), stride=(1, 1, 1))
    (act2): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (up2): ConvTranspose3d(64, 32, kernel_size=(2, 2, 2), stride=(2, 2, 2))
  (dec1): ResidualBlock3d(
```



residual blocks instead of plain conv blocks dropout in deeper layers same U-Net structure same 2-channel input 1-channel output

TRAIN / TEST SPLIT FOR ARCHITECTURE EXPERIMENT

```
# ========================================
# CELL 10: TRAIN / TEST SPLIT
# ========================================
TRAIN_IDS = [
    "00713519","01092669","01386182","01406050","01722254",
    "01406050_1","01563167","01581148_2","01581148_4","01668999",
    "00516779","00749499","00832617","00890265","00890509",
    "00907490","00952337","00967329","00967939","01137872",
]

TEST_IDS = [
    "00960975","00981157","01003818","01019493",
    "01036141","01083481","01135143"
]

print("Train patients:", len(TRAIN_IDS))
print("Test patients:", len(TEST_IDS))
```




```
Train patients: 20
Test patients: 7
```

DATASET FOR PATCH TRAINING

```
# ========================================
# CHECK REAL FILENAMES INSIDE ONE PATIENT
# ========================================
import os

pid = "00713519"
patient_dir = os.path.join(PREP_ROOT, pid)

print("Folder:", patient_dir)
print("\nFiles:")
for f in os.listdir(patient_dir):
    print(f)
```

```
Folder: /content/drive/MyDrive/ms_preprocessed/00713519

Files:
followup_0p5_norm.nii.gz
baseline_0p5_registered_to_followup_norm.nii.gz
label_0p5_in_followup_space.nii.gz
baseline_0p5_norm_padded.nii.gz
followup_registered_0p5_norm_padded.nii.gz
label_0p5_padded.nii.gz
```



```
# ========================================
# CELL 11: PATCH DATASET
# ========================================
from torch.utils.data import Dataset, DataLoader
import random

PATCH_SIZE = (64, 64, 64)
SAMPLES_PER_EPOCH = 800
POS_FRACTION = 0.5

class MultiPatientBalancedPatchDataset(Dataset):
    def __init__(self, patient_ids, patch_size=(64,64,64), samples_per_epoch=800, pos_fraction=0.5):
        self.patient_ids = patient_ids
        self.patch_size = patch_size
        self.samples_per_epoch = samples_per_epoch
        self.pos_fraction = pos_fraction
```

```
        self.data = {}
        for pid in patient_ids:
            b, f, y = load_preprocessed(pid)
            self.data[pid] = (b, f, y)

    def __len__(self):
        return self.samples_per_epoch

    def __getitem__(self, idx):
        pid = random.choice(self.patient_ids)
        b, f, y = self.data[pid]

        ph, pw, pd = self.patch_size
        H, W, D = y.shape

        use_positive = (random.random() < self.pos_fraction) and (np.sum(y) > 0)

        if use_positive:
            lesion_voxels = np.argwhere(y > 0)
            center = lesion_voxels[np.random.randint(len(lesion_voxels))]
            cz, cy, cx = center[0], center[1], center[2]

            z1 = max(0, min(cz - ph//2, H - ph))
            y1 = max(0, min(cy - pw//2, W - pw))
            x1 = max(0, min(cx - pd//2, D - pd))
        else:
            z1 = np.random.randint(0, H - ph + 1)
            y1 = np.random.randint(0, W - pw + 1)
            x1 = np.random.randint(0, D - pd + 1)

        b_patch = b[z1:z1+ph, y1:y1+pw, x1:x1+pd]
        f_patch = f[z1:z1+ph, y1:y1+pw, x1:x1+pd]
        y_patch = y[z1:z1+ph, y1:y1+pw, x1:x1+pd]

        x_patch = np.stack([b_patch, f_patch], axis=0)
        y_patch = np.expand_dims(y_patch, axis=0)

        return torch.tensor(x_patch, dtype=torch.float32), torch.tensor(y_patch, dtype=torch.float32)

train_ds = MultiPatientBalancedPatchDataset(
    TRAIN_IDS,
    patch_size=PATCH_SIZE,
    samples_per_epoch=SAMPLES_PER_EPOCH,
    pos_fraction=POS_FRACTION
)

train_loader = DataLoader(train_ds, batch_size=2, shuffle=True, num_workers=2, pin_memory=True)

print("Dataset ready.")
```

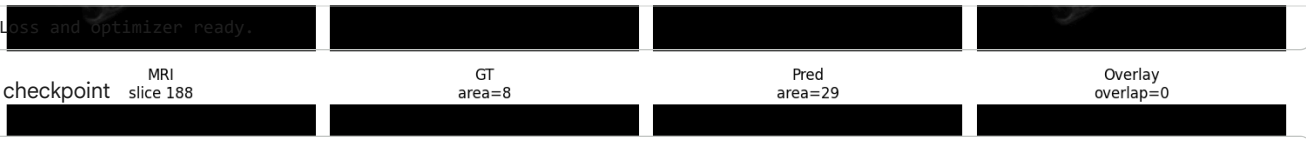
```
Dataset ready.
```

CELL 12 — LOSS + OPTIMIZER

```
# ========================================
# CELL 12: LOSS, OPTIMIZER, METRIC
# ========================================
import torch.nn as nn

def soft_dice_loss(logits, targets, eps=1e-8):
    probs = torch.sigmoid(logits)
    num = 2 * torch.sum(probs * targets) + eps
    den = torch.sum(probs) + torch.sum(targets) + eps
    return 1 - num / den

def dice_bce_loss(logits, targets):
    dice = soft_dice_loss(logits, targets)
    bce = nn.BCEWithLogitsLoss()(logits, targets)
    return dice + bce

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4,
    weight_decay=1e-6
)
```

```
print("Loss and optimizer ready.")
```

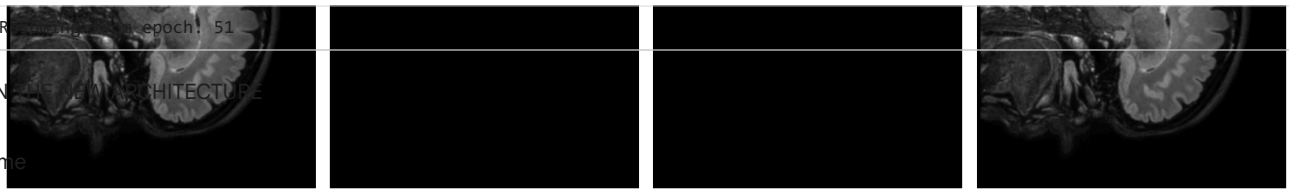
```
Loss and optimizer ready.
```



Load checkpoint

```
# ========================================
# CELL 12.5: RESUME FROM SAVED CHECKPOINT
# ========================================
CKPT_PATH = os.path.join(
    PREP_ROOT,
    "_checkpoints",
    "residual_unet_train20_lr3e-4_wd1e-6",
    "epoch_50.pt"   # latest saved checkpoint before epoch 73
)

ckpt = torch.load(CKPT_PATH, map_location=device)

model.load_state_dict(ckpt["model_state_dict"])
optimizer.load_state_dict(ckpt["optimizer_state_dict"])

start_epoch = ckpt["epoch"] + 1
print("Resuming from epoch:", start_epoch)
```

```
Resuming from epoch: 51
```



TRAIN

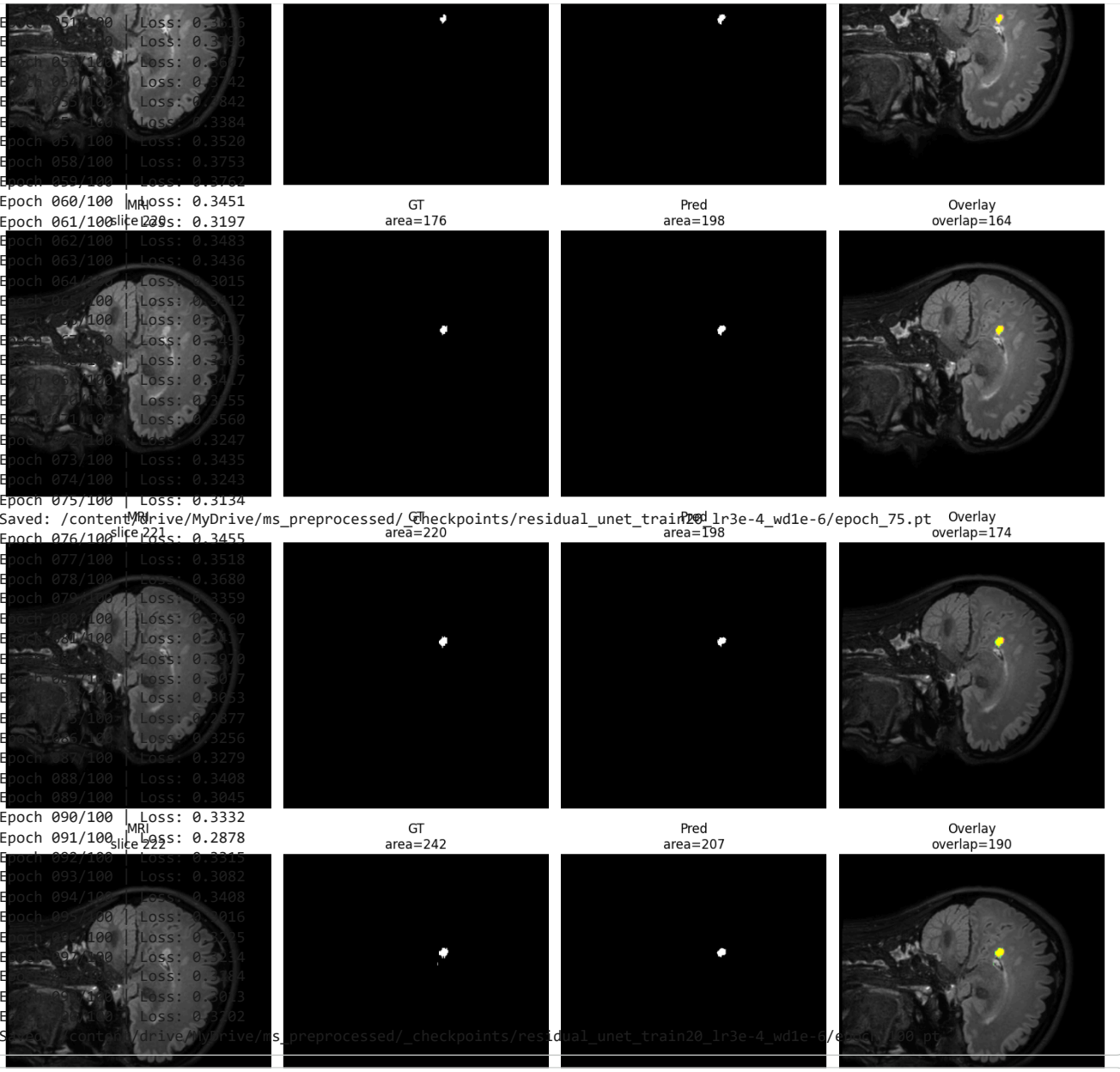
ARCHITECTURE

Resume

```
# ========================================
# CELL 13: RESUME TRAINING
# ========================================
import os

SAVE_DIR = os.path.join(
    PREP_ROOT,
    "_checkpoints",
    "residual_unet_train20_lr3e-4_wd1e-6"
)
os.makedirs(SAVE_DIR, exist_ok=True)

MAX_EPOCHS = 100
SAVE_EVERY = 25

history = []

for epoch in range(start_epoch, MAX_EPOCHS + 1):
    model.train()
    running_loss = 0.0

    for x, y in train_loader:
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        optimizer.zero_grad()
        logits = model(x)
        loss = dice_bce_loss(logits, y)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    mean_loss = running_loss / len(train_loader)
    history.append((epoch, mean_loss))

    print(f"Epoch {epoch:03d}/{MAX_EPOCHS} | Loss: {mean_loss:.4f}")

    if epoch % SAVE_EVERY == 0 or epoch == MAX_EPOCHS:
        path = os.path.join(SAVE_DIR, f"epoch_{epoch}.pt")
        torch.save({
            "epoch": epoch,
            "model_state_dict": model.state_dict(),
```

```
            "optimizer_state_dict": optimizer.state_dict(),
            "loss": mean_loss
        }, path)
        print("Saved:", path)
```

```
Epoch 060/100 | Loss: 0.3451
Epoch 061/100 | Loss: 0.3197
Epoch 062/100 | Loss: 0.3483
Epoch 063/100 | Loss: 0.3436
Epoch 073/100 | Loss: 0.3435
Epoch 074/100 | Loss: 0.3243
Epoch 075/100 | Loss: 0.3134
Saved: /content/drive/MyDrive/ms_preprocessed/_checkpoints/residual_unet_train20_lr3e-4_wd1e-6/epoch_75.pt
Epoch 076/100 | Loss: 0.3455
Epoch 077/100 | Loss: 0.3518
Epoch 078/100 | Loss: 0.3680
Epoch 087/100 | Loss: 0.3279
Epoch 088/100 | Loss: 0.3408
Epoch 089/100 | Loss: 0.3045
Epoch 090/100 | Loss: 0.3332
Epoch 091/100 | Loss: 0.2878
Epoch 093/100 | Loss: 0.3082
Epoch 094/100 | Loss: 0.3408
```




```
# ========================================
# VIEW ONE PATIENT AT SPECIFIC EPOCH
# ========================================
BEST_EPOCH = 25
PID = "00981157"

CKPT_PATH = os.path.join(SAVE_DIR, f"epoch_{BEST_EPOCH}.pt")

# Load trained model
best_model = load_residual_model_checkpoint(CKPT_PATH)

# Run patient summary
summary, b, f, gt, prob, pred = summarize_patient(PID, best_model, threshold=0.5)

print(summary)
```

```
# ========================================
# CELL 13: TRAIN NEW ARCHITECTURE
# ========================================
import os

SAVE_DIR = os.path.join(
```

```
    PREP_ROOT,
    "_checkpoints",
    "residual_unet_train20_lr3e-4_wd1e-6"
)
os.makedirs(SAVE_DIR, exist_ok=True)

MAX_EPOCHS = 100     # start with 100 first
SAVE_EVERY = 25

history = []

for epoch in range(1, MAX_EPOCHS + 1):
    model.train()
```